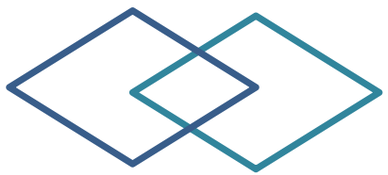


第10章

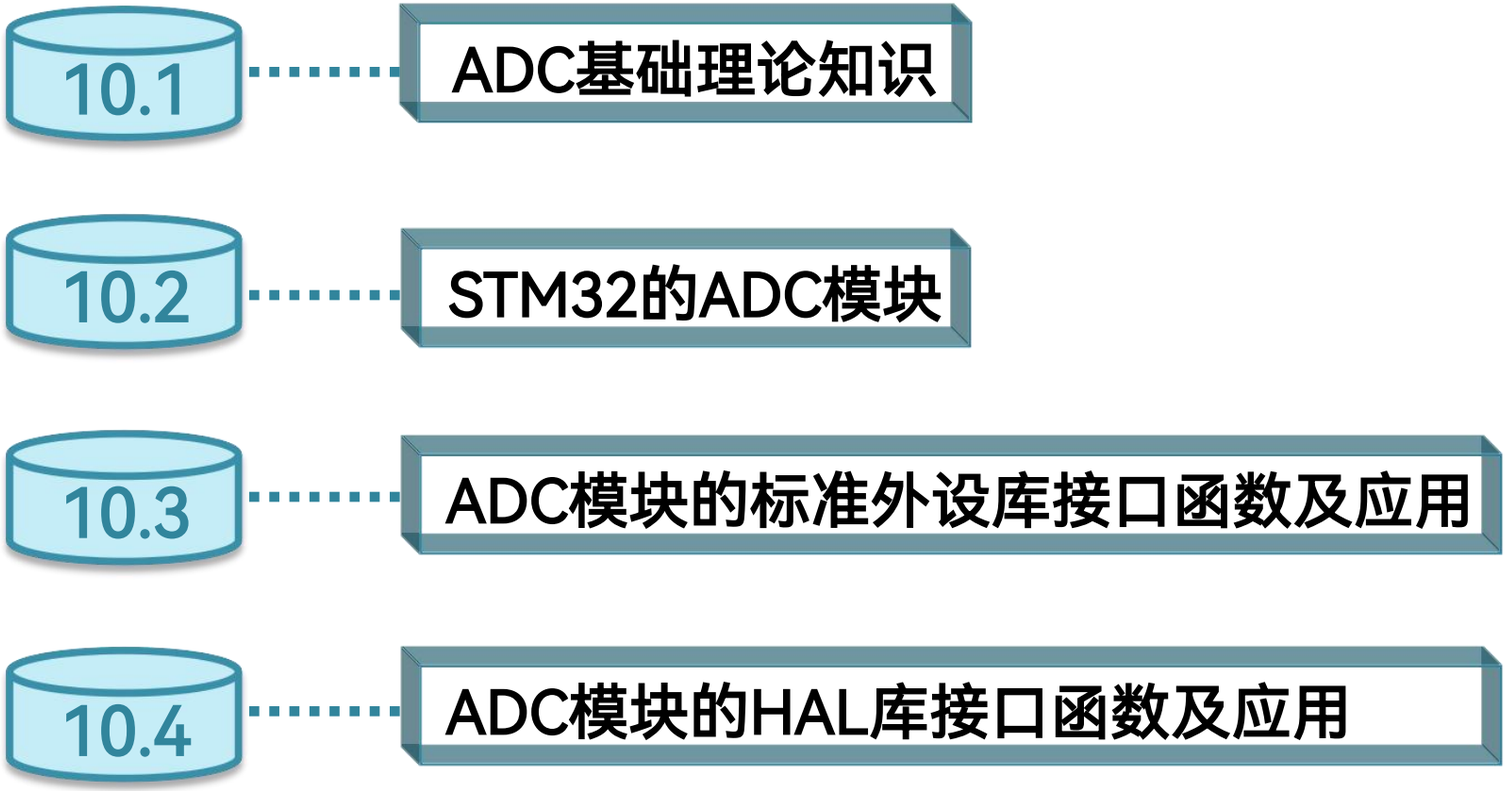
ADC



本章知识与能力要求

- 理解和掌握ADC的转换过程、主要技术参数；
- 理解STM32的ADC内部结构；
- 熟悉STM32标准外设库v3.5.0和HAL库中有关ADC的库函数；
- 掌握基于标准外设库的ADC开发；
- 掌握基于HAL库的ADC开发。

第10章 ADC


10.1
ADC基础理论知识
10.2
STM32的ADC模块
10.3
ADC模块的标准外设库接口函数及应用
10.4
ADC模块的HAL库接口函数及应用

10.1 ADC基础理论知识

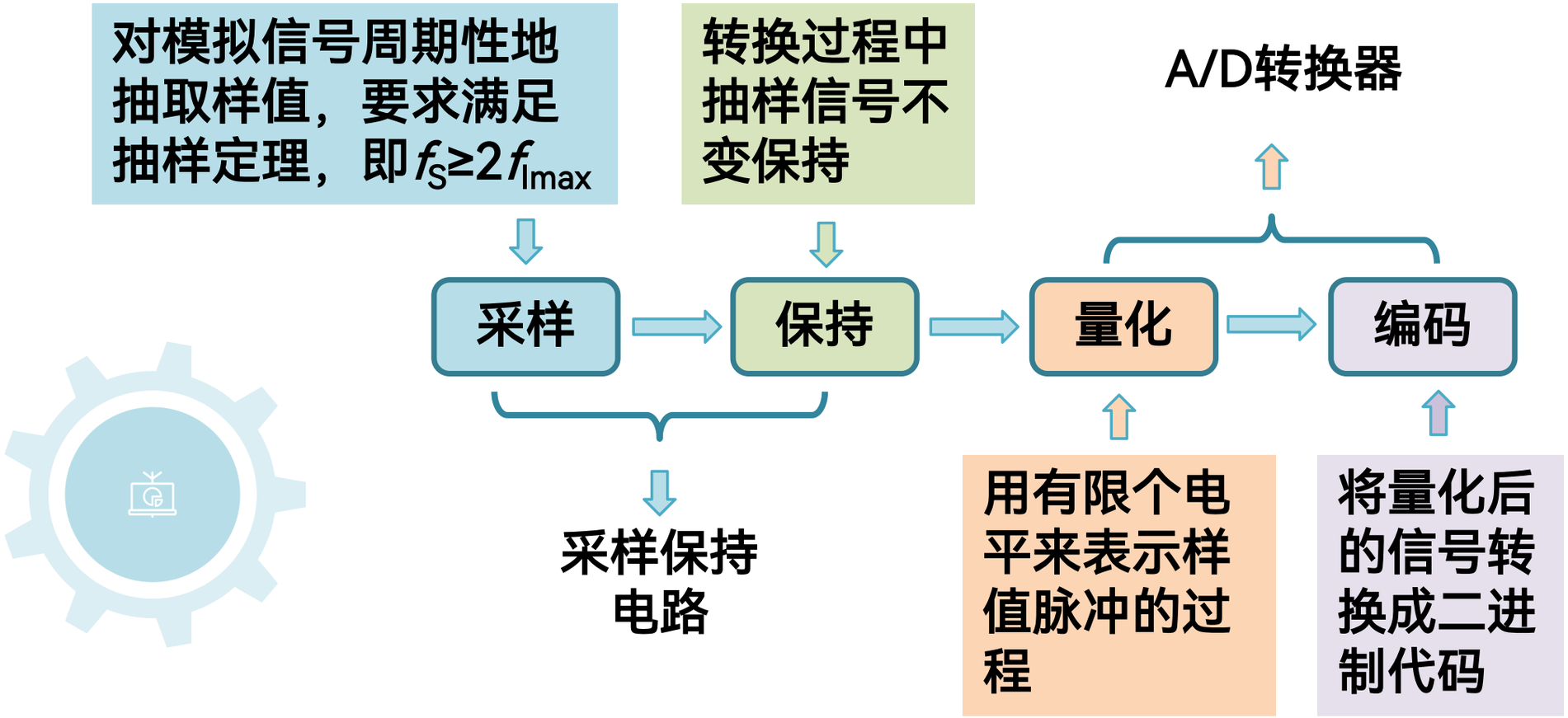
目 录


10.1.1 A/D转换过程


10.1.2 A/D转换的主要技术参数

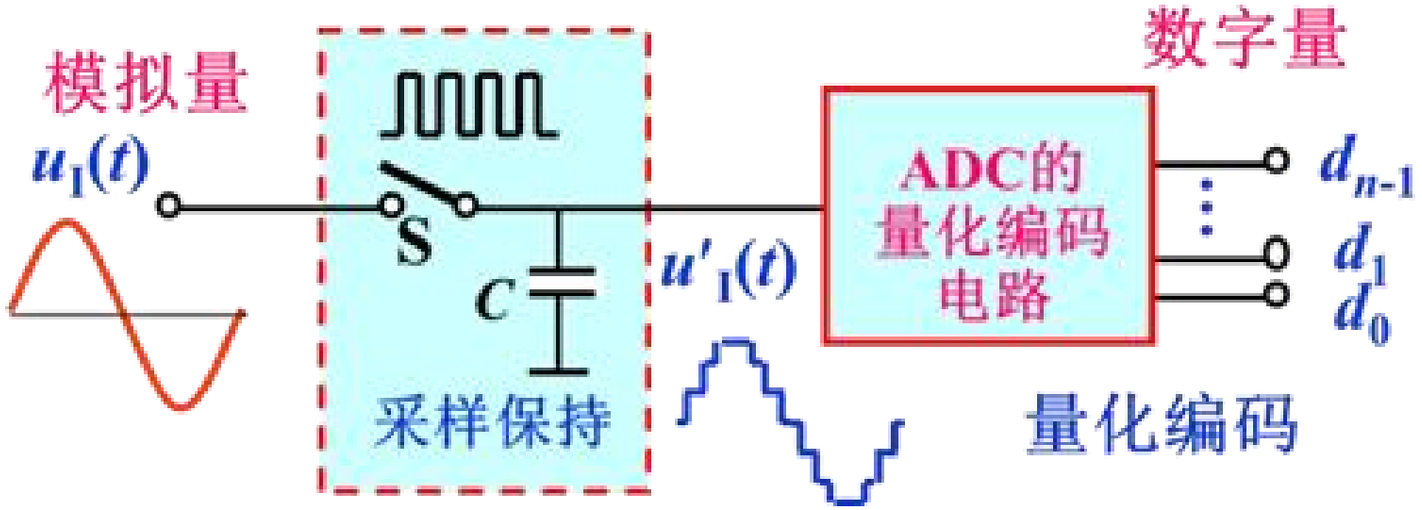
10.1.1 A/D转换过程

ADC（Analog to Digital Converter）即模数转换器，用来将模拟信号转换为数字信号。



10.1.1 A/D转换过程

ADC(Analog to Digital Converter）即模数转换器，用来将模拟信号转换为数字信号。



A/D转换过程

10.1.2 A/D转换的主要技术参数

- 技术参数
  - 转换精度 —— 用分辨率和转换误差
  - 转换速度 —— 转换时间或转换速率

分辨率

A/D转换器对输入模拟量微小变化的分辨能力，通常用二进制数的有效位表示。

在最大输入电压一定时，位数越多，量化单位越小，误差越小，分辨率越高

位数	级数	分辨率(精度)	说明
8	$2^8 = 256$	39.1 mV	8位、12位ADC常用于工控领域、语音处理领域
10	$2^{10} = 1024$	9.77 mV	
12	$2^{12} = 4096$	2.44 mV	
14	$2^{14} = 16384$	0.61 mV	
16	$2^{16} = 65536$	0.15 mV	16位或更高的ADC常用于医学领域中微弱生理信号的采集

10.1.2 A/D转换的主要技术参数

转换时间/转换速率

转换时间是指从转换控制信号到来到A/D转换器输出端得到稳定的数字量所需要的时间。转换时间的倒数即为**转换速率**。

转换时间与A/D转换器类型有关：

- **双积分型A/D转换器**的转换时间一般在几十毫秒数量级，属于低速A/D转换器；
- **逐次比较型**在几十微秒，属于中速A/D转换器；
- **并行比较型**一般在几十纳秒。

转换误差

由于A/D转换器的有限分辨率而引起的误差，反映A/D转换器实际输出数字量和理想输出之间的差异。

通常为1个或半个最小数字量的模拟变化量，表示为1LSB、1/2LSB。



10.2 STM32的ADC

集成有三个12位ADC逐次逼近型的A/D转换器



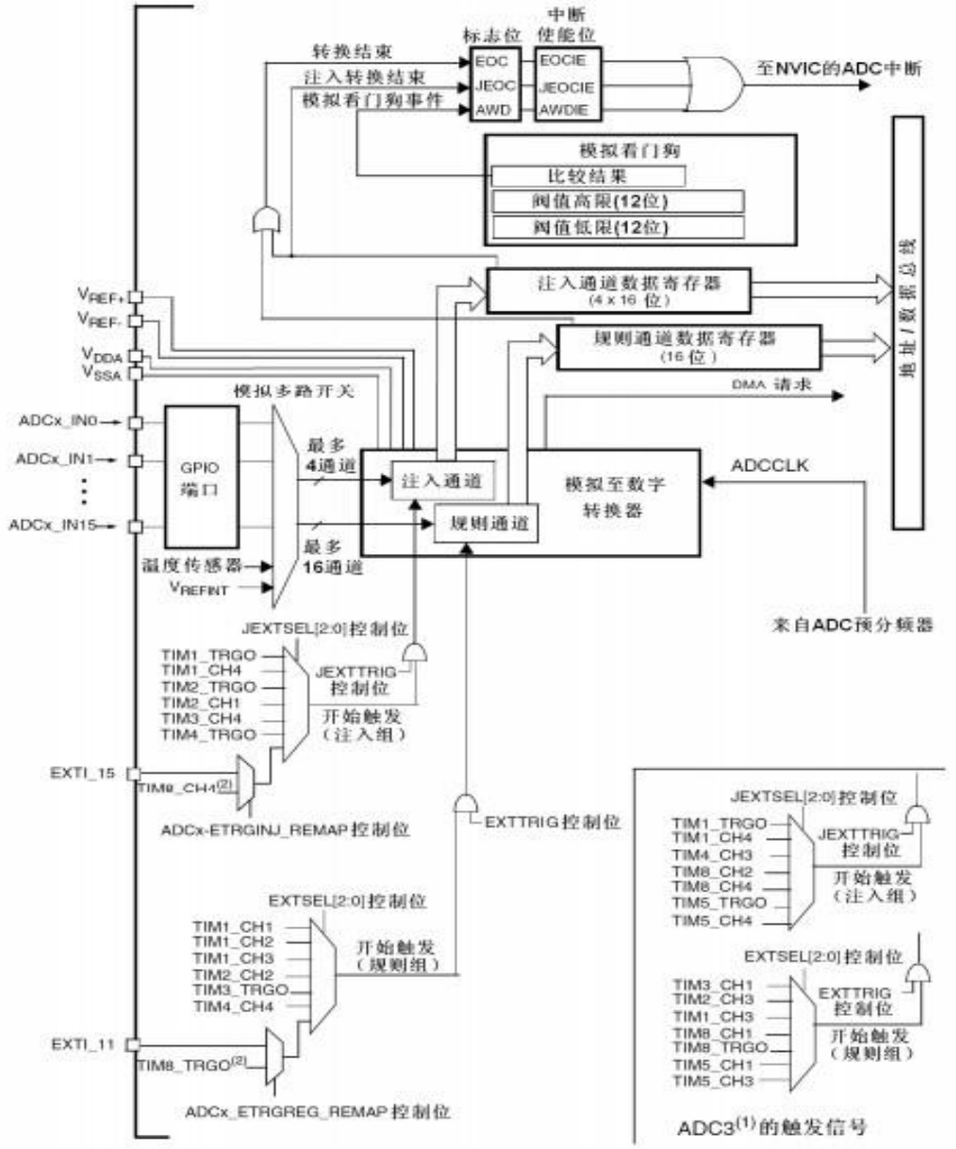
多达18个通道，可实现16个外部模拟输入通道和2个内部信号源的A/D转换

转换结果是一个12位的二进制数，可以以左对齐或右对齐两种方式存储在16位数据寄存器中

各通道的A/D转换可采用单次、连续、扫描或间断模式执行

10.2 STM32的ADC

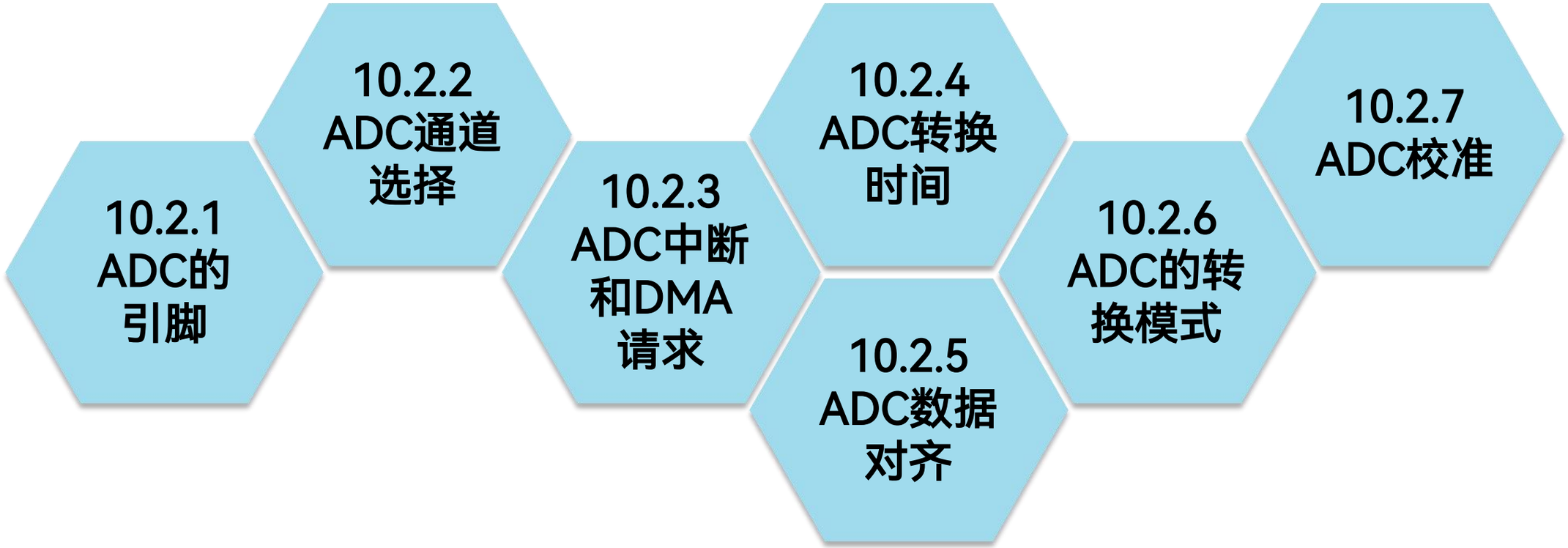
- ADC
  - ADC引脚
  - 模拟多路开关
  - ADC时钟
  - 注入或规则通道
  - 中断



10.2 STM32的ADC

信号输入	触发信号	A/D转换	数据保存
输入信号经ADC的输入通道ADCx_IN0~ADCx_IN15送入模拟至数字转换器功能模块。	软件触发或EXTI外部触发或定时器触发。将硬件触发源分为两类：规则通道和注入通道。	在ADC时钟ADCCLK的驱动下，对输入通道的信号进行A/D转换，即完成采样—量化—编码等工作。	A/D转换结束后，将转换结果的12位数据以左对齐或右对齐的方式保存到16位的寄存器中。

10.2 STM32的ADC模块



10.2.1 ADC的引脚

ADCx_IN0~ADCx_IN15为ADC的输入信号通道，即为ADC的输入GPIO引脚，ADC的GPIO引脚均为复用功能，使用时需配置为模拟输入模式。

ADC1还有两个内部通道：温度传感器通道和V_{REFINT}。

通道	ADC1（复用引脚）	ADC2（复用引脚）	ADC3（复用引脚）
通道0(ADC_Channel_0)	PA0	PA0	PA0
通道1(ADC_Channel_1)	PA1	PA1	PA1
通道2(ADC_Channel_2)	PA2	PA2	PA2
通道3(ADC_Channel_3)	PA3	PA3	PA3
通道4(ADC_Channel_4)	PA4	PA4	PF6
通道5(ADC_Channel_5)	PA5	PA5	PF7
通道6(ADC_Channel_6)	PA6	PA6	PF8
通道7(ADC_Channel_7)	PA7	PA7	PF9
通道8(ADC_Channel_8)	PB0	PB0	PF10
通道9(ADC_Channel_9)	PB1	PB1	
通道10(ADC_Channel_10)	PC0	PC0	PC0
通道11(ADC_Channel_11)	PC1	PC1	PC1
通道12(ADC_Channel_12)	PC2	PC2	PC2
通道13(ADC_Channel_13)	PC3	PC3	PC3
通道14(ADC_Channel_14)	PC4	PC4	

10.2.2 ADC通道选择

按组进行转换的模式，可以由程序设置实现对多个模拟通道自动地进行逐个采样转换，STM32分为两种组转换模式：规则组和注入组。

规则通道组最多允许16个规则通道进行转换

规则组

可通过编程来设定规则通道的数量n，n最多可设定为16。

转换结果保存在同一个16位的规则通道数据寄存器ADC_DR中，同时，EOC标志被置位，产生相应的中断或DMA请求。

10.2.2 ADC通道选择

规则组

例如：设定n=3，规则通道组由1、2、3、5、6、7、9和11通道组成：

第一次触发时，转换的序列为1、2和3通道；

第二次触发，转换的序列为5、6和7通道；

第三次触发，转换的序列为9和11通道，然后产生EOC事件；

第四次触发，转换的序列为1、2和3通道，以此类推。

10.2.2 ADC通道选择

最多允许4个通道进行转换，并且对应有4个注入通道寄存器用来存放注入通道的转换结果，因此注入通道组没有DMA请求。

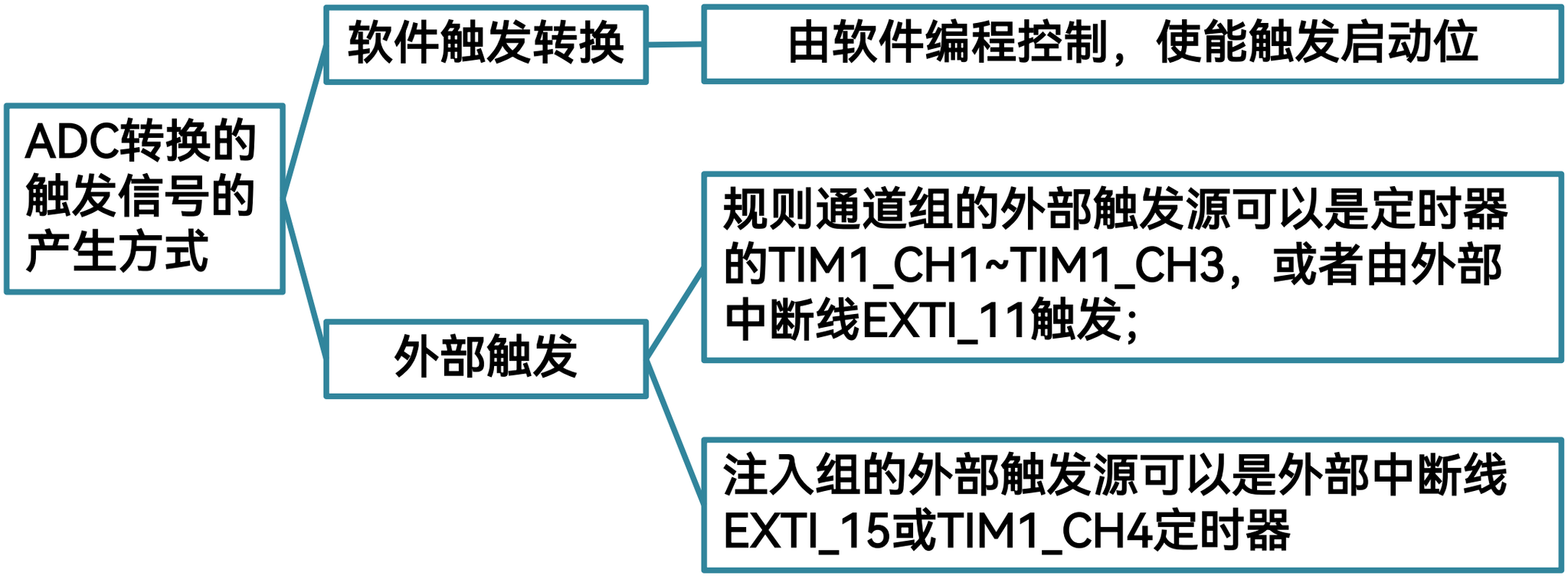
注入通道组的触发转换序列和规则通道一样



转换结果保存数据寄存器ADC_JDRx中，同时产生ADC注入转换结束事件，即JEOC标志被置位，产生相应的中断。

规则通道组转换好比是程序的正常执行，注入通道组的转换则好比是一个中断处理程序

10.2.3 ADC中断和DMA请求



10.2.3 ADC中断和DMA请求

ADC在每个通道转换完成后，可产生相应的中断请求。

对于规则通道，如果ADC_CR1寄存器的EOCIE位被置“1”，则会产生EOC中断；

对于注入通道，如果ADC_CR1寄存器的JEOCIE位被置“1”，则会产生JEOC中断；

ADC1和ADC3的规则通道转换完成后还可产生DMA请求。

10.2.3 ADC中断和DMA请求

ADC中断事件主要有三个，ADC_IT_EOC中断针对规则通道，ADC_IT_JEOC中断针对注入通道。

中断事件	事件标志	使能控制位
规则组转换结束中断 ADC_IT_EOC	EOC中断 (End of Coversion)	EOCIE
注入组转换结束中断 ADC_IT_JEOC	JEOC中断 (End of injected Coversion)	JEOCIE
模拟看门狗中断 ADC_IT_AWD	AWD中断 (Analog WatchDOG)	AWDIE

10.2.3 ADC中断和DMA请求

DMA请求

适用情况

只有ADC1和ADC3能产生DMA请求，ADC2转换数据可以在双ADC模式中使用ADC1的DMA请求

使用原因

规则通道的转换只有一个数据寄存器ADC_DR，每个通道转换完成后，将覆盖以前的数据，所以，可以使用DMA方式处理数据及时地将已完成转换的数据读出。

应用

在每次产生转换结束事件EOC标志后，DMA控制器会把保存在ADC_DR寄存器中的规则组通道的转换数据传送到SRAM中(用户指定的目标地址)

10.2.4 ADC转换时间

STM32的ADC的采样时间可选择为采样周期的1.5、7.5、13.5、28.5、41.5、55.5、71.5或239.5倍。

ADC总的转换时间可以表示为：

ADC总的转换时间=采样时间+12.5个周期

举例

若采样时间为1.5个周期，PCLK2=72MHz，且ADCCLK为APB2的6分频，即12MHz，则

总的转换时间=（1.5+12.5）/12MHz=1.17us

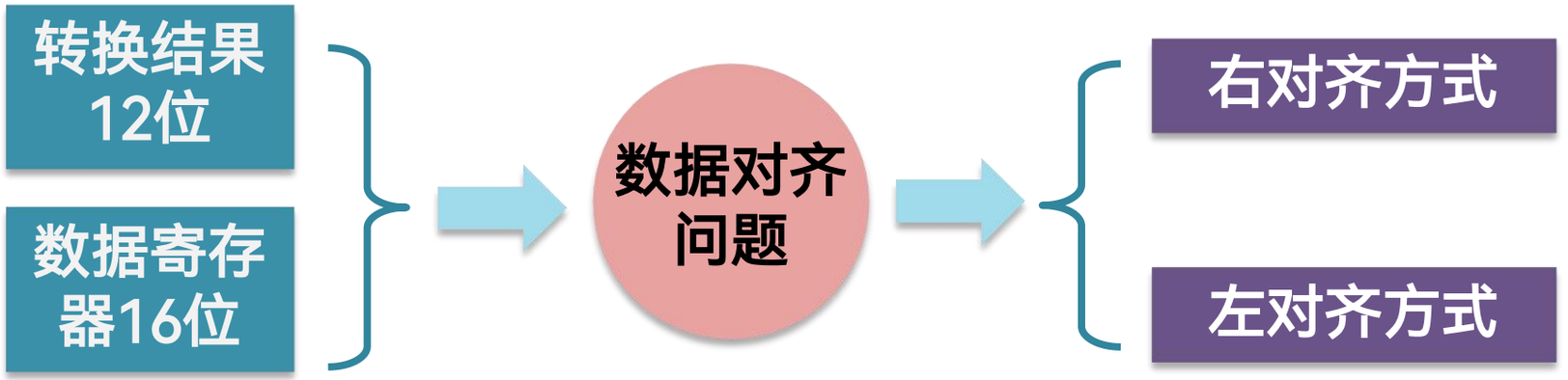
如果ADCCLK=14MHz，采样时间为1.5个周期，则

总的转换时间=（1.5+12.5）/14MHz=1us

注意

ADC的时钟ADCCLK最大不能超过14MHz，因此STM32F103系列微控制器的ADC最短转换时间为1us。

10.2.5 ADC数据对齐



一般建议采用**右对齐**，因为数据传输一般是从**最低位**开始(也即右边开始），如果数据位是从高位开始传输的，可以采用左对齐方式。

10.2.6 ADC的转换模式



ADC只执行一次转换

当前ADC转换一结束就会立即启动下一个转换

扫描一组模拟通道：

- 单次模式：扫描完本组所有通道后，ADC自动停止；
- 连续模式：扫描完本组所有通道后，再从第一个通道继续扫描。

用于多个通道的规则组和注入组

10.2.6 ADC的转换模式

间断模式

以规则组为例，若被转换的通道为0、1、2、3、6、7、9、10，n=3，则

第一次触发：进行转换的通道序列为0、1、2；

第二次触发：进行转换的通道序列为3、6、7；

第三次触发：进行转换的通道序列为9、10，并产生EOC事件；

第四次触发：进行转换的通道序列0、1、2。

在间断模式下，转换也有其特殊的规则：

- 当以间断模式转换一个规则组时，转换序列结束后不自动从头开始；
- 当所有子组被转换完成，下一次触发启动第一个子组的转换；
- 规则组和注入组不能同时设置为间断模式。

10.2.7 ADC校准

校准目的

A/D转换过程需要一定的转换时间，对ADC进行校准可以减少因内部电容的变化而导致的误差。

校准实现

STM32的ADC具有内置的自校准模式，ADC在上电后至少2个ADC时钟周期才能开始校准

校准要求

建议每次上电后都执行一次ADC校准，以保证采集数据的准确性。

10.3 ADC模块的标准外设库接口函数及应用



10.3.1 ADC标准外设库接口函数



10.3.2 ADC标准外设库配置过程



10.3.3 ADC标准外设库应用实例

10.3.1 ADC标准外设库接口函数

分布在stm32f10x_adc.c文件和stm32f10x_adc.h文件中，stm32f10x_adc.h头文件定义了ADC初始化结构体，并声明了相应的库函数。

ADC初始化结构体ADC_InitTypeDef用于配置ADC的相关参数。

ADC初始化结构体

```
typedefstruct
{
uint32_t ADC_Mode; //ADC工作模式
FunctionalState ADC_ScanConvMode;
  //是否采用扫描模式
FunctionalState ADC_ContinuousConvMode;
  //是否采用连续转换模式
uint32_t ADC_ExternalTrigConv; //外部触发启动
uint32_t ADC_DataAlign; //ADC的数据对齐方式
uint8_t ADC_NbrOfChannel; //ADC的通道数目
}ADC_InitTypeDef;
```

10.3.1 ADC标准外设库接口函数

ADC初始化结构体成员及其取值

结构体成员	取值范围
ADC_Mode （ADC工作模式）	ADC_Mode_Independent（独立模式） ADC_Mode_RegInjecSimult（同步规则和同步注入模式） ADC_Mode_RegSimult_AlterTrig（同步规则和交替触发模式） ADC_Mode_InjecSimult_FastInterl（同步注入和快速交替模式） ADC_Mode_InjecSimult_SlowInterl（同步注入和慢速交替模式） ADC_Mode_InjecSimult（同步注入模式） ADC_Mode_RegSimult（同步规则模式） ADC_Mode_FastInterl（快速交替模式） ADC_Mode_SlowInterl（慢速交替模式） ADC_Mode_AlterTrig（交替触发模式）
ADC_ScanConvMode （是否采用扫描模式）	ENABLE（开启扫描模式） DISABLE（不开启扫描模式）
ADC_ContinuousConvMode （是否采用连续转换模式）	ENABLE（开启连续转换模式） DISABLE（单次转换模式）

10.3.1 ADC标准外设库接口函数

ADC初始化结构体成员及其取值

结构体成员	取值范围
ADC_ExternalTrigConv （外部触发启动）	ADC_ExternalTrigConv_T1_CC1（定时器1的捕获比较1） ADC_ExternalTrigConv_T1_CC2（定时器1的捕获比较2） ADC_ExternalTrigConv_T1_CC3（定时器1的捕获比较3） ADC_ExternalTrigConv_T2_CC2（定时器2的捕获比较2） ADC_ExternalTrigConv_T3_TRGO（定时器3的RGO） ADC_ExternalTrigConv_T4_CC4（定时器4的捕获比较4） ADC_ExternalTrigConv_Ext_IT11_TIM8_TRGO（外部中断线11触发） ADC_ExternalTrigConv_None（转换由软件触发而不是外部触发启动）
ADC_DataAlign （ADC的数据对齐方式）	ADC_DataAlign_Right（ADC数据右对齐） ADC_DataAlign_Left（ADC数据左对齐）
ADC_NbrOfChannel （ADC的通道数目）	ADC_Channel_0 ~ ADC_Channel_17

10.3.1 ADC标准外设库接口函数

库函数1

void ADC_DeInit(ADC_TypeDef* ADCx);

功能：将外设ADCx的全部寄存器复位为初始值。

实例：ADC_DeInit(ADC1); //复位ADC1

库函数2

void ADC_Init(ADC_TypeDef* ADCx, ADC_InitTypeDef* ADC_InitStruct);

功能：初始化ADCx。

实例：ADC_Init(ADC1,&ADC_InitStructure); //初始化ADC1

库函数3

void ADC_Cmd(ADC_TypeDef* ADCx, FunctionalStateNewState);

功能：使能或失能指定的ADCx。

实例：ADC_Cmd(ADC1, ENABLE); //使能ADC1

10.3.1 ADC标准外设库接口函数

库函数4

void ADC_DMACmd(ADC_TypeDef* ADCx, FunctionalStateNewState);

功能：使能或失能指定的ADCx的DMA请求。

实例：ADC_DMACmd(ADC1, ENABLE); //使能ADC1的DMA请

库函数5

void ADC_ITConfig(ADC_TypeDef* ADCx, uint16_t ADC_IT, FunctionalStateNewState);

功能：使能或失能指定的ADCx的中断。

例：ADC_ITConfig(ADC1, ADC_IT_EOC|ADC_IT_AWD, ENABLE);//使能ADC1的EOC和AWD中断

- 参数ADC_IT取值
  - ADC_IT_EOC — EOC中断屏蔽
  - ADC_IT_AWD — AWDOG中断屏蔽
  - ADC_IT_JEOC — JEOC中断屏蔽

10.3.1 ADC标准外设库接口函数

库函数6

void ADC_ResetCalibration(ADC_TypeDef* ADCx);

功能：重置指定ADCx的校准寄存器。

实例：ADC_ResetCalibration(ADC1); //重置ADC1的校准寄存器

库函数7

FlagStatus ADC_GetResetCalibrationStatus(ADC_TypeDef* ADCx);

功能：获取ADCx的重置校准寄存器状态。

实例：

```
FlagStatus ADC_Status; //定义状态变量ADC_Status
ADC_Status=ADC_GetResetCalibrationStatus(ADC1);
    //获取ADC1的重置校准寄存器的状态
```

10.3.1 ADC标准外设库接口函数

库函数8

void ADC_StartCalibration(ADC_TypeDef* ADCx);

功能：开始指定ADCx的校准。

实例：ADC_StartCalibration(ADC2); //开始ADC2的校准

库函数9

FlagStatus ADC_GetCalibrationStatus(ADC_TypeDef* ADCx);

功能：获取ADCx的校准状态。

实例：ADC_GetCalibrationStatus (ADC2); //获取ADC2的校准状态

库函数10

void ADC_SoftwareStartConvCmd(ADC_TypeDef* ADCx, FunctionalStateNewState);

功能：使能或者失能指定的ADCx的软件转换启动功能。

实例：ADC_SoftwareStartConvCmd(ADC1, ENABLE);//开启ADC1软件转换启动功能

10.3.1 ADC标准外设库接口函数

库函数11

FlagStatus ADC_GetSoftwareStartConvStatus(ADC_TypeDef* ADCx);

功能：获取ADCx软件转换启动状态。

实例：FlagStatus ADC_Status; //定义状态变量ADC_Status

ADC_Status = ADC_GetSoftwareStartConvStatus(ADC1);

//获取ADC1的软件转换启动状态

库函数12

void ADC_RegularChannelConfig(ADC_TypeDef* ADCx, uint8_t ADC_Channel, uint8_t Rank, uint8_t ADC_SampleTime);

功能：设置指定ADCx的规则组通道，设置它们的转化顺序和采样时间。

实例：

ADC_RegularChannelConfig(ADC1, ADC_Channel_2, 1,ADC_SampleTime_7Cycles5);

10.3.1 ADC标准外设库接口函数

库函数13

uint16_t ADC_GetConversionValue(ADC_TypeDef* ADCx);

功能：返回最近一次ADCx规则组的转换结果。

实例：

```
uint16_t ADC_ConvertedValue;
    //定义变量ADC_ConvertedValue用于存放ADC转换结果
ADC_ConvertedValue = ADC_GetConversionValue(ADC1);
    //获取ADC1转换结果
```

10.3.1 ADC标准外设库接口函数

库函数14

FlagStatus ADC_GetFlagStatus(ADC_TypeDef* ADCx, uint8_t ADC_FLAG);

功能：检查ADCx的相应标志位是否置“1”。

实例：

```
FlagStatus ADC_Status;//定义状态变量ADC_Status
ADC_Status = ADC_GetFlagStatus(ADC1, ADC_FLAG_EOC);
```

ADC_FLAG的值

ADC_AnalogWatchdog	说明
ADC_FLAG_AWD	模拟看门狗标志位
ADC_FLAG_EOC	转换结束标志位
ADC_FLAG_JEOC	注入组转换结束标志位
ADC_FLAG_JSTRT	注入组转换开始标志位
ADC_FLAG_STRT	规则组转换开始标志位

10.3.1 ADC标准外设库接口函数

库函数15

void ADC_ClearFlag(ADC_TypeDef* ADCx, uint8_t ADC_FLAG);

功能：清除ADCx的相应标志位。

实例：ADC_ClearFlag(ADC1,ADC_FLAG_STRT);

//清除ADC1的规则组转换开始标志位

库函数16

ITStatus ADC_GetITStatus(ADC_TypeDef* ADCx, uint16_t ADC_IT);

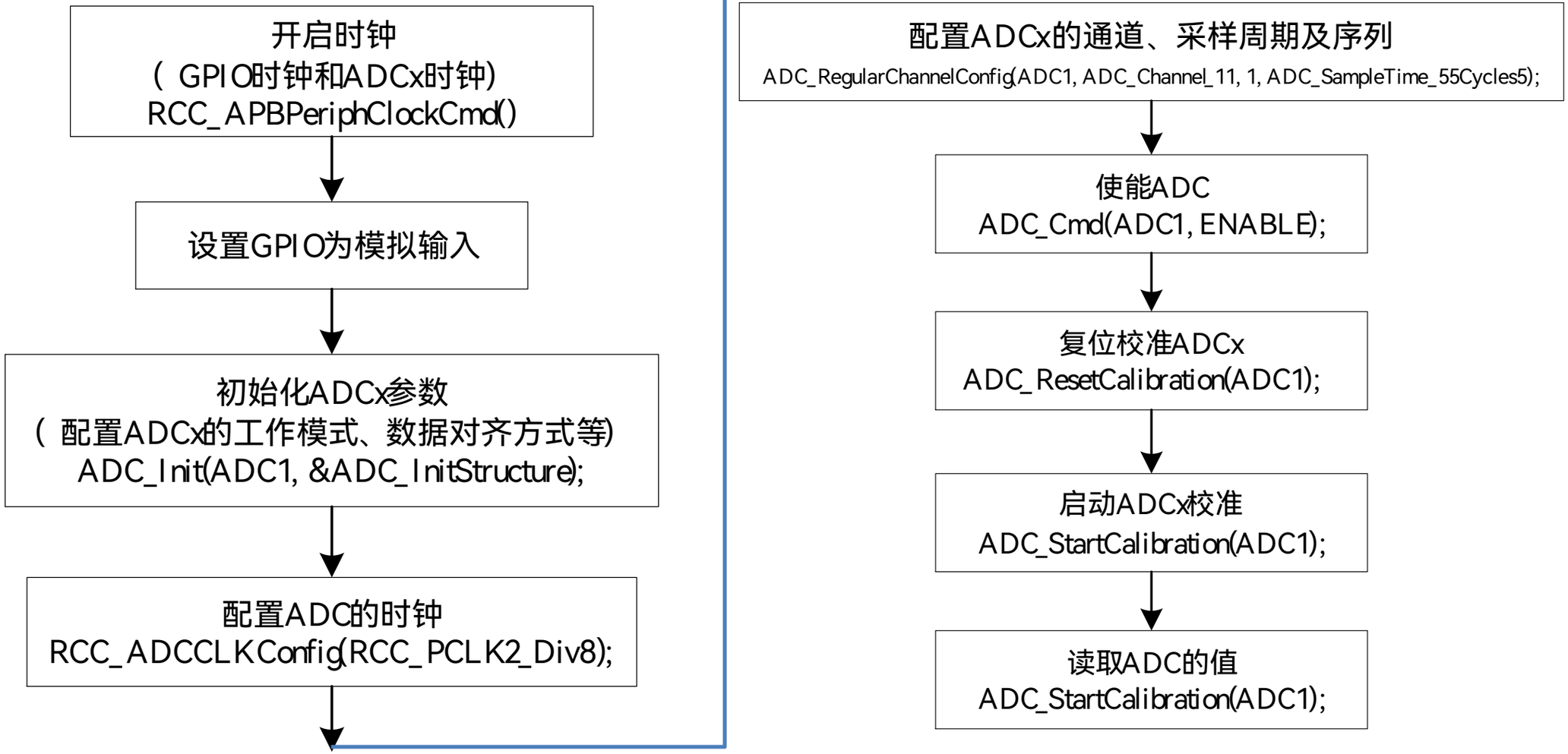
功能：获取ADCx的ADC_IT中断的状态。

实例：

```
ITStatus ADC_Status;//定义状态变量ADC_Status
ADC_Status = ADC_GetITStatus(ADC1, ADC_IT_EOC);
    //获取ADC1的EOC中断屏蔽状态
```

10.3.2 ADC标准外设库配置过程

ADC的配置流程图



10.3.2 ADC标准外设库配置过程

步骤1：开启PC端口和ADC1的时钟，设置PC1为模拟输入

- 使能PC端口和ADC1的时钟
- 使用RCC_APB2PeriphClockCmd()函数
- 设置PC1的输入方式为模拟输入。

步骤2：复位ADC1，同时设置ADC1的分频因子

- 复位：将ADC1的全部寄存器重新设置为默认值；
- 设置分频因子：确保ADC1的时钟不超过14MHz，此处将分频因子设为6，ADC1的时钟为72/6=12MHz；
- 库函数的实现方式为：

```
RCC_ADCCLKConfig(RCC_PCLK2_Div
6);
```

- ADC时钟复位的库函数为：

10.3.2 ADC标准外设库配置过程

步骤3：初始化ADC1参数，设置ADC1的工作模式及其它参数

- 初始化参数涉及设置单次转换模式、触发方式和数据对齐方式等参数，使用库函数ADC_Init()来实现。
- Void ADC_Init(ADC_TypeDef * ADCx，ADC_InitTypeDef * ADC_InitStruct);
- 第一个参数用于指定具体的ADCx（x=1,2,3），第二个参数是个结构体。

ADC1初始化配置实例

```
ADC_InitStructure.ADC_Mode = ADC_Mode_Independent;
ADC_InitStructure.ADC_ScanConvMode = ENABLE;
ADC_InitStructure.ADC_ContinuousConvMode = ENABLE;
ADC_InitStructure.ADC_ExternalTrigConv = ADC_ExternalTrigConv_None;
ADC_InitStructure.ADC_DataAlign = ADC_DataAlign_Right;
ADC_InitStructure.ADC_NbrOfChannel = 1;
ADC_Init(ADC1, &ADC_InitStructure);
```

10.3.2 ADC标准外设库配置过程

步骤4：使能ADC并校准

- 进行复位校准和ADC校准；
- 复位校准的标准外设库函数为ADC_ResetCalibration(ADC1)
- ADC校准的标准外设库函数为ADC_StartCalibration(ADC1)。
- ADC已准备就绪后软件开启ADC1转换的标准外设库函数为：ADC_SoftwareStartConvCmd(ADC1,ENABLE);

步骤5：读取ADC的值

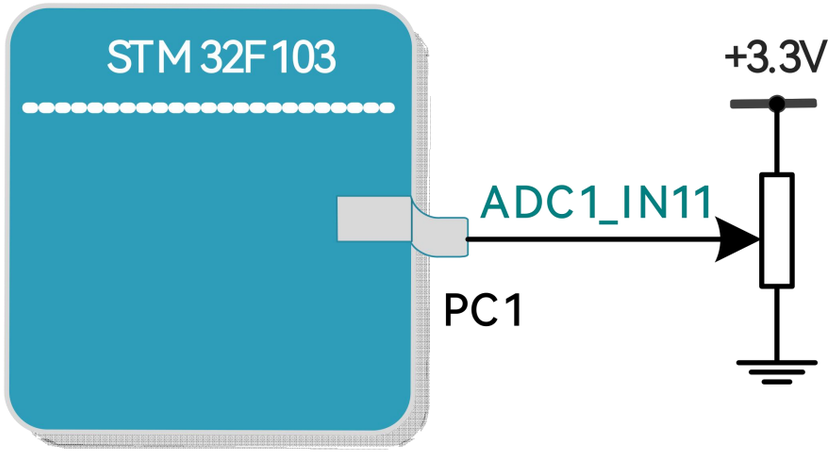
- 读取ADC转换结果的标准外设库函数为：ADC_GetConversionValue(ADC1);

10.3.3 ADC标准外设库应用实例

功能

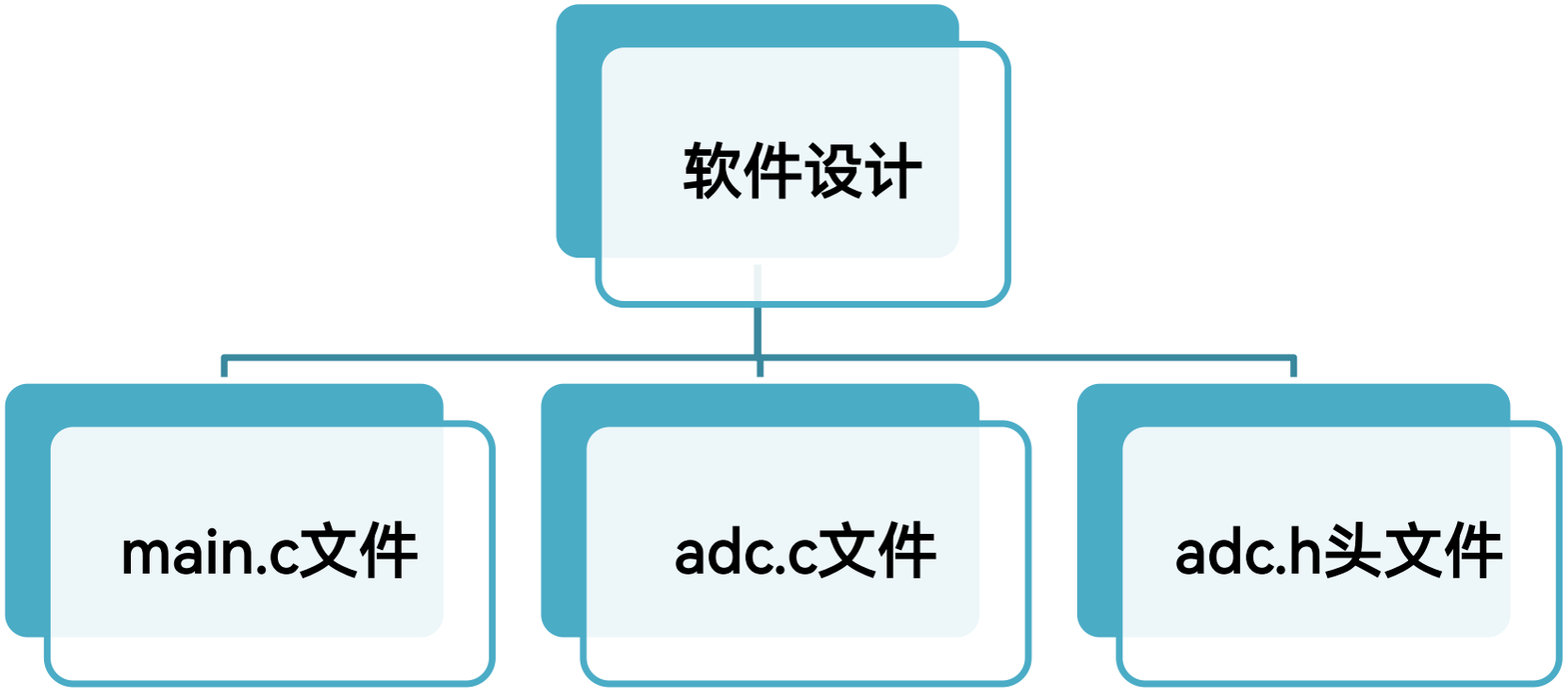
通过PC1（ADC1通道11）采集外部电压值，经A/D转换后，采用DMA方式将该数字量和由该数字量对应计算获得的电压值通过串口显示出来。

硬件设计



STM32的PC1引脚连接到一个滑动变阻器上采集滑动变阻器的模拟输入电压，经A/D转换后采用DMA方式传输到内存，并通过串口显示。

10.3.3 ADC标准外设库应用实例



10.3.3 ADC标准外设库应用实例

1．新建adc.c文件

```
#include "adc.h"
#define  ADC1_DR_Address    ((u32)0x40012400+0x4c)
   //ADC1的数据寄存器ADC1_DR的地址
_ _IO  uint16_t  ADC_Converted_Value;
  //定义一个变量用于存放ADC转换结果
static void ADC1_GPIO_Config(void)
{   GPIO_InitTypeDef GPIO_InitStructure;
    RCC_AHBPeriphClockCmd(RCC_AHBPeriph_DMA1,
ENABLE);
       //开启DMA1的时钟
      /*开启 ADC1 and GPIOC时钟*/
  RCC_APB2PeriphClockCmd(RCC_APB2Periph_ADC1 |
 RCC_APB2Periph_GPIOC, ENABLE);
    /*配置PC1为模拟输入*/
   GPIO_InitStructure.GPIO_Pin = GPIO_Pin_1;
   GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AIN;
   GPIO_Init(GPIOC, &GPIO_InitStructure);
 }
```

```
   static void ADC1_Mode_Config(void)
{
   DMA_InitTypeDef DMA_InitStructure;
   ADC_InitTypeDef ADC_InitStructure;
      /* DMA channel1 configuration */
   DMA_DeInit(DMA1_Channel1);
   DMA_InitStructure.DMA_PeripheralBaseAddr =
   ADC1_DR_Address; //ADC1地址
   DMA_InitStructure.DMA_MemoryBaseAddr = (u32)&
   ADC_Converted_Value;//内存地址
   DMA_InitStructure.DMA_DIR = DMA_DIR_PeripheralSRC;
      //外设作为数据传输的来源
    DMA_InitStructure.DMA_BufferSize = 1;//DMA传输1个数
据
    DMA_InitStructure.DMA_PeripheralInc =
   DMA_PeripheralInc_Disable;//外设地址固定
    DMA_InitStructure.DMA_MemoryInc =
   DMA_MemoryInc_Disable;  //内存地址固定
```

10.3.3 ADC标准外设库应用实例

1．新建adc.c文件

```
DMA_InitStructure.DMA_PeripheralDataSize=DMA_PeripheralDataSize_Half
Word;
    //传输单位为半字
DMA_InitStructure.DMA_MemoryDataSize =
DMA_MemoryDataSize_HalfWord;
DMA_InitStructure.DMA_Mode = DMA_Mode_Circular; //循环传输DMA模式
DMA_InitStructure.DMA_Priority = DMA_Priority_High;
DMA_InitStructure.DMA_M2M = DMA_M2M_Disable;
DMA_Init(DMA1_Channel1, &DMA_InitStructure); //配置DMA通道1
DMA_Cmd(DMA1_Channel1, ENABLE); //使能DMA1通道1
    /* ADC1参数配置 */
ADC_InitStructure.ADC_Mode = ADC_Mode_Independent; //独立ADC模式
ADC_InitStructure.ADC_ScanConvMode = DISABLE ; //禁止扫描模式
```

10.3.3 ADC标准外设库应用实例

1．新建adc.c文件

```
ADC_InitStructure.ADC_ExternalTrigConv = ADC_ExternalTrigConv_None; //不用外部触发
ADC_InitStructure.ADC_DataAlign = ADC_DataAlign_Right;//数据对齐方式为右对齐
ADC_InitStructure.ADC_NbrOfChannel = 1;          //要转换的通道个数
ADC_Init(ADC1, &ADC_InitStructure);
RCC_ADCCLKConfig(RCC_PCLK2_Div8); //配置ADC时钟，为PCLK2的8分频，即9MHz
    /*配置ADC1的通道11为55.5个采样周期，序列为1 */
ADC_RegularChannelConfig(ADC1, ADC_Channel_11, 1,
ADC_SampleTime_55Cycles5);
ADC_DMACmd(ADC1, ENABLE); //使能ADC1 的DMA
ADC_Cmd(ADC1, ENABLE); //使能ADC1
```

10.3.3 ADC标准外设库应用实例

```
while(ADC_GetResetCalibrationStatus(ADC1));
    //等待校准寄存器复位完成
ADC_StartCalibration(ADC1);
   //启动ADC1校准
while(ADC_GetCalibrationStatus(ADC1));
    //等待校准完成
ADC_SoftwareStartConvCmd(ADC1, ENABLE);
     //软件触发ADC转换
}
```

1．新建adc.c文件

```
/*
*函数名：ADC1_Init(void)
*功能说明：ADC1初始化，通过调用ADC1_GPIO_Config()和ADC1_Mode_Config()函数来实现
*输入：无
*输出：无
*/
void ADC1_Init(void)
{
        ADC1_GPIO_Config();
        ADC1_Mode_Config();
}
```

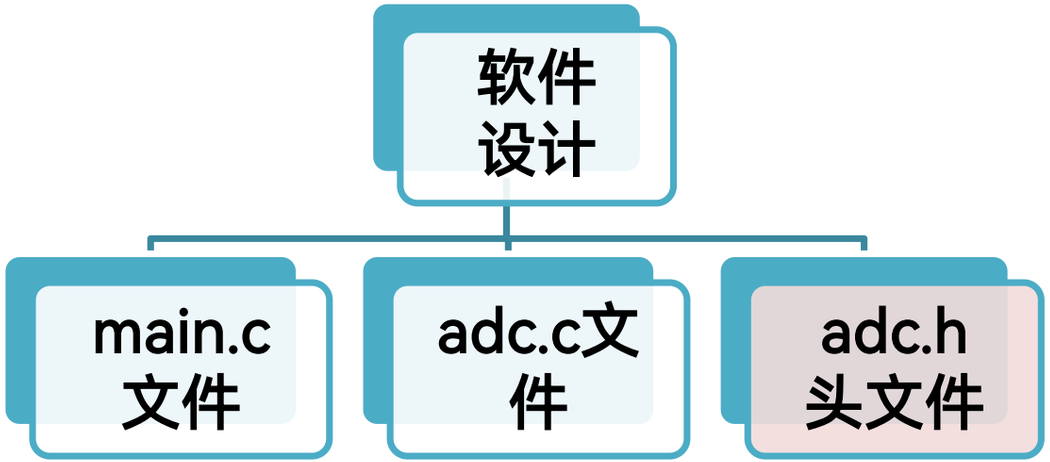
10.3.3 ADC标准外设库应用实例

2．新建与adc.c对应的头文件adc.h

```
#ifndef__ADC_H
#define__ADC_H

#include "stm32f10x.h"
void ADC1_Init(void);

#endif
```



10.3.3 ADC标准外设库应用实例

3．应用程序实现(main.c文件)

```
#include "stm32f10x.h"
#include "usart.h"
#include "adc.h"
#include "systick.h"
//ADC1转换的电压值通过DMA方式传送到SRAM
extern  __IO uint16_t ADC_Converted_Value;
//定义一个局部变量，用于保存转换计算后的电压值
float  ADC_ConvertedValueLocal;
```

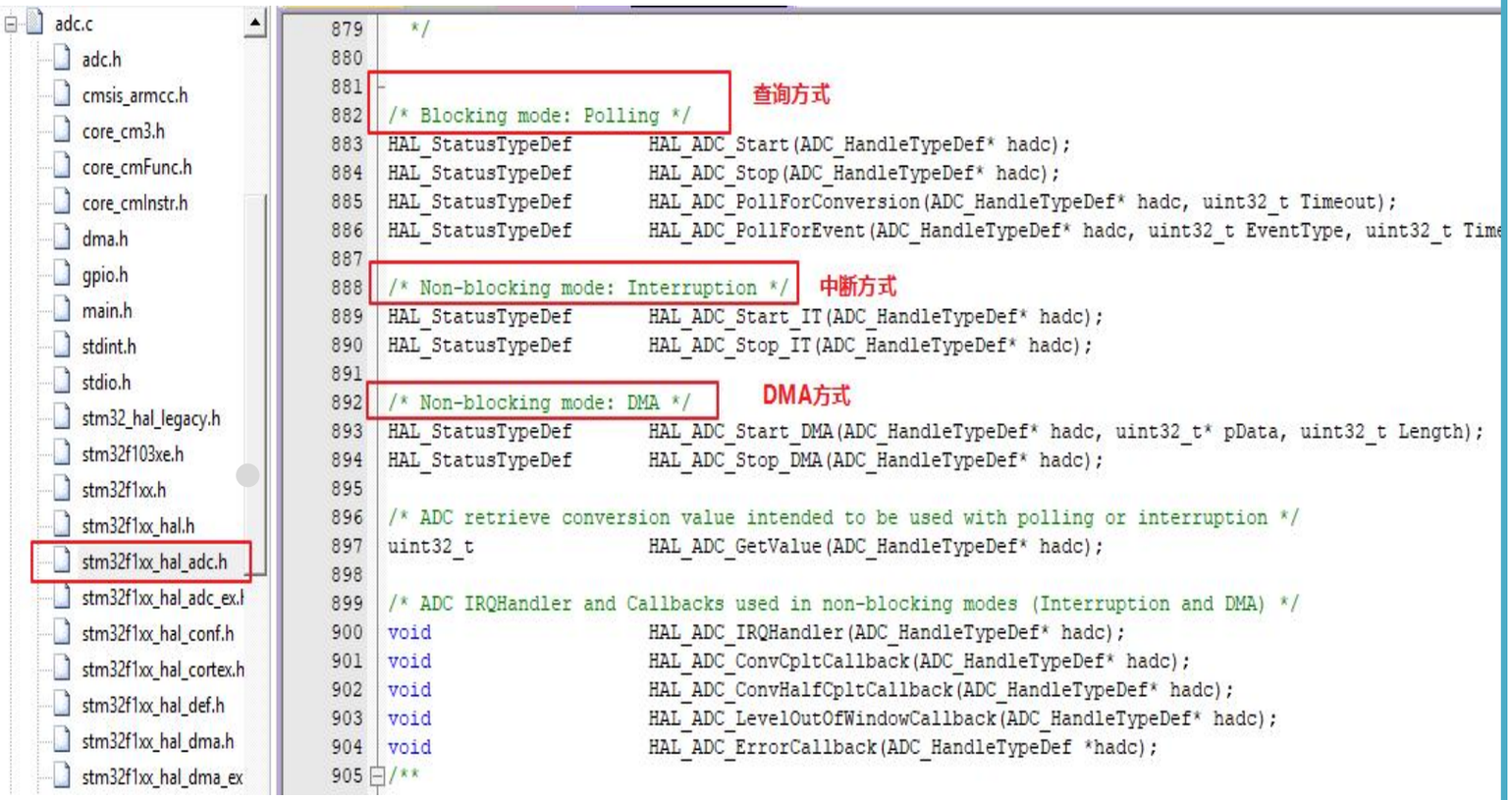
```
int main(void)
{
   ADC1_Init();
   printf("\n\r ADC 模数转换测试实例\r");
   while (1)
    {
      ADC_ConvertedValueLocal =(float) ADC_Converted_Value
      /4096*3.3;
         //计算数字量对应的电压值
      printf("\r\n 当前电压采集值的16进制 = 0x%04X \r\n",
      ADC_Converted_Value);
      printf("\r\n 当前电压的模拟值 = %f V
      \r\n",ADC_ConvertedValueLocal);
      Delay_ms(500);
         //调用SysTick定时器实现精确延时，请参考本书第9章SysTick一节
    }
}
```

10.4 ADC模块的HAL库接口函数及应用

10.4.1 ADC的HAL库接口函数

10.4.2 ADC的HAL库应用实例

10.4.1 ADC的HAL库接口函数



- 定义在stm32f1xx_hal_adc.c源文件
- 在stm32f1xx_hal_adc.h头文件中可以查看HAL库中DMA库函数的声明以及相关的结构体定义和串口一样
- 可以通过轮询、中断和DMA三种方式进行操作。

10.4.1 ADC的HAL库接口函数

ADC的HAL库常用接口函数

初始化及复位函数

- HAL_ADC_Init(ADC_HandleTypeDef* hadc);
  功能描述：ADC初始化函数
- HAL_ADC_DeInit(ADC_HandleTypeDef *hadc);
  功能描述：ADC复位函数
- void HAL_ADC_MspInit(ADC_HandleTypeDef* hadc);
  功能描述：ADC初始化配置函数，该函数被HAL库内部调用

10.4.1 ADC的HAL库接口函数

ADC的HAL库常用接口函数

类型		函数及功能描述
引脚功能操作函数	轮询方式	HAL_ADC_Start(ADC_HandleTypeDef* hadc); 功能描述：启动ADC转换
		HAL_ADC_Stop(ADC_HandleTypeDef* hadc); 功能描述：停止ADC转换
		HAL_ADC_PollForConversion(ADC_HandleTypeDef* hadc, uint32_t Timeout); 功能描述：等待转换结束，采用超时管理机制
	中断方式	HAL_ADC_Start_IT(ADC_HandleTypeDef* hadc); 功能描述：中断方式下启动ADC 转换
		HAL_ADC_Stop_IT(ADC_HandleTypeDef* hadc); 功能描述：中断方式下停止ADC 转换
	DMA方式	HAL_ADC_Start_DMA(ADC_HandleTypeDef* hadc, uint32_t* pData, uint32_t Length); 功能描述：DMA方式下启动ADC转换
		HAL_ADC_Stop_DMA(ADC_HandleTypeDef* hadc); 功能描述：DMA方式下停止ADC转换
		uint32_t HAL_ADC_GetValue(ADC_HandleTypeDef* hadc); 功能描述：轮询或中断方式下获取ADC转换值

10.4.1 ADC的HAL库接口函数

ADC的HAL库常用接口函数

类型	函数及功能描述
中断处理函数(回调函数)	void HAL_ADC_IRQHandler(ADC_HandleTypeDef* hadc); 功能描述：ADC中断处理函数
	void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc); 功能描述：ADC中断回调函数，用户在该函数内编写实际的中断处理程序
	void HAL_ADC_ConvHalfCpltCallback(ADC_HandleTypeDef* hadc); 功能描述：ADC转换一半时调用的中断回调函数
外设配置函数	HAL_ADC_ConfigChannel(ADC_HandleTypeDef* hadc, ADC_ChannelConfTypeDef* sConfig); 功能描述：ADC配置函数
外设状态函数	uint32_t HAL_ADC_GetState(ADC_HandleTypeDef* hadc); 功能描述：获取ADC转换状态函数
	uint32_t HAL_ADC_GetError(ADC_HandleTypeDef *hadc); 功能描述：获取ADC错误函数

10.4.1 ADC的HAL库接口函数

1．启动ADC转换函数HAL_ADC_Start()

函数原型

```
HAL_StatusTypeDef  HAL_ADC_Start(ADC_HandleTypeDef*
hadc);
```

使用范例

```
 if(HAL_ADC_Start(&hadc1) != HAL_OK)
{   Error_Handler();
 }
    /*等待转换结束*/
HAL_ADC_PollForConversion(&hadc1, 10);
if(HAL_ADC_GetState(&hadc1) ==
HAL_ADC_STATE_EOC_REG)
{
/*获取转换结果*/
ADCx_ConvertedValue = HAL_ADC_GetValue(&hadc1);
}
```

10.4.1 ADC的HAL库接口函数

2．启动ADC中断转换函数HAL_ADC_Start_IT()

函数原型

```
HAL_StatusTypeDef  HAL_ADC_Start_IT(ADC_HandleTypeDef*
hadc);
```

使用范例

```
 if(HAL_ADC_StartIT(&hadc1) != HAL_OK)
{
    Error_Handler();
}
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef*
hadc)
{
    /* 获取ADC转换结果 */
   ADCx_ConvertedValue =
HAL_ADC_GetValue(&hadc1);
```

10.4.1 ADC的HAL库接口函数

3．启动ADC的DMA转换函数 HAL_ADC_Start_DMA()

函数原型

```
HAL_StatusTypeDef HAL_ADC_Start_DMA(ADC_HandleTypeDef* hadc, uint32_t* pData, uint32_t Length);
```

使用范例

```
if(HAL_ADC_Start_DMA(&hadc1,&ADCx_ConvertedValue,1) != HAL_OK)
{ Error_Handler();
}
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc)
{
/* DMA传输结束，指示灯LED亮 */
HAL_GPIO_WritePin(GPIOE, GPIO_PIN_5, GPIO_PIN_RESET);
}
```

10.4.2 ADC的HAL库应用实例

功能描述

内部有一个温度传感器，测量范围为：-40℃~125℃，测量精度范围为±1.5℃，用来测量芯片内部的温度，连接在ADC1_IN16的输入通道上。

本实例将采集到的温度数据通过串口发送到PC机。

Reading the temperature

To use the sensor:

1. Select the ADCx_IN16 input channel.
2. Select a sample time of 17.1 µs
3. Set the TSVREFE bit in the *ADC control register 2 (ADC_CR2)* to wake up the temperature sensor from power down mode.
4. Start the ADC conversion by setting the ADON bit (or by external trigger).
5. Read the resulting V_{SENSE} data in the ADC data register
6. Obtain the temperature using the following formula:

   Temperature (in °C) = {(V_{25} - V_{SENSE}) / Avg_Slope} + 25.

   Where,

   V_{25} = V_{SENSE} value for 25° C and

   Avg_Slope = Average Slope for curve between Temperature vs. V_{SENSE} (given in mV/° C or µV/ °C).

   Refer to the Electrical characteristics section for the actual values of V_{25} and Avg_Slope.

STM32内部温度传感器说明

10.4.2 HAL库ADC应用实例

5.3.21 Temperature sensor characteristics

Table 64. TS characteristics

Symbol	Parameter	Min	Typ	Max	Unit
T_L	V_{SENSE} linearity with temperature	-	±1	±2	°C
Avg_Slope	Average slope	4.0	4.3	4.6	mV/°C
V_{25}	Voltage at 25 °C	1.34	1.43	1.52	V
$t_{START}^{(1)}$	Startup time	4	-	10	µs
$T_{S_temp}^{(2)(1)}$	ADC sampling time when reading the temperature	-	-	17.1	µs

1. Guaranteed by design.
2. Shortest sampling time can be determined in the application by multiple iterations.

V25为温度传感器在25℃时的输出电压，典型值为1.43V；

Avg_Slope为温度传感器输出电压与温度曲线的平均斜率的关联参数，典型值为4.3mV/℃

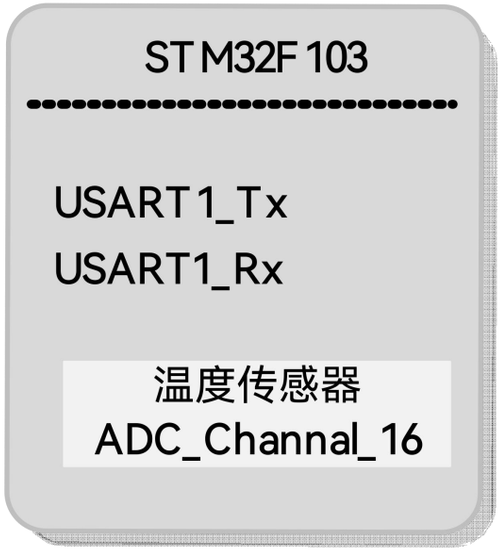
10.4.2 HAL库ADC应用实例

硬件设计



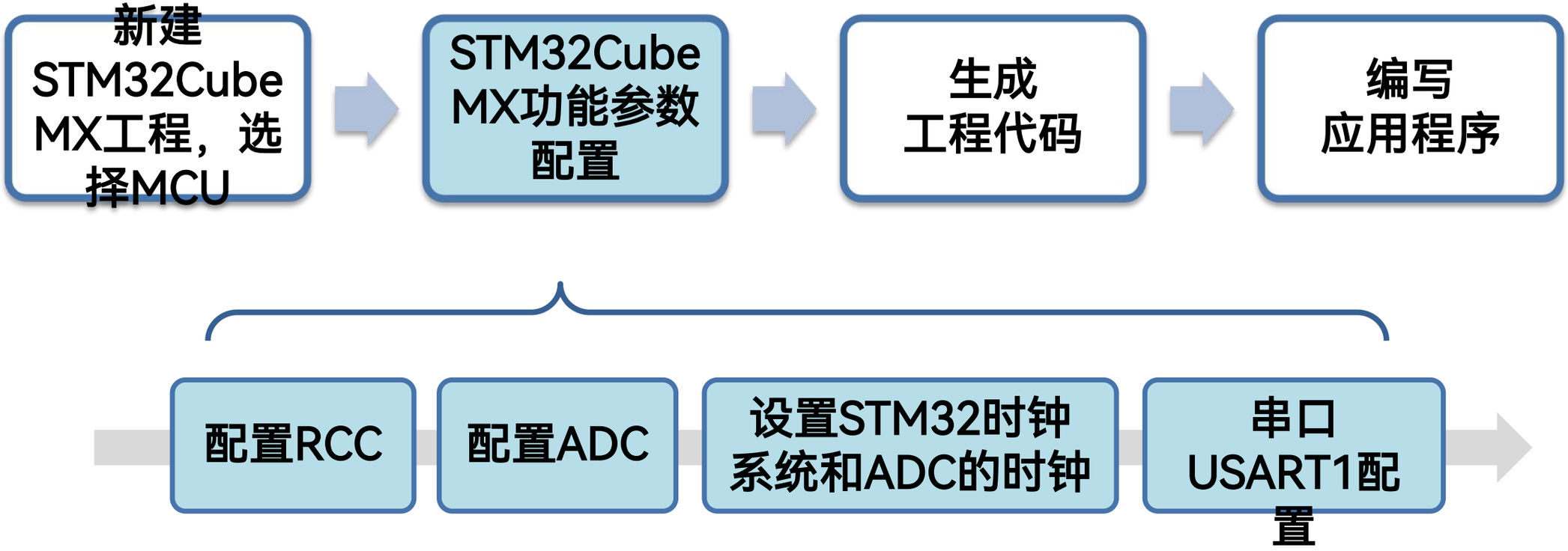
PC机需安装USB转串口
模块CH341驱动

DMA传输



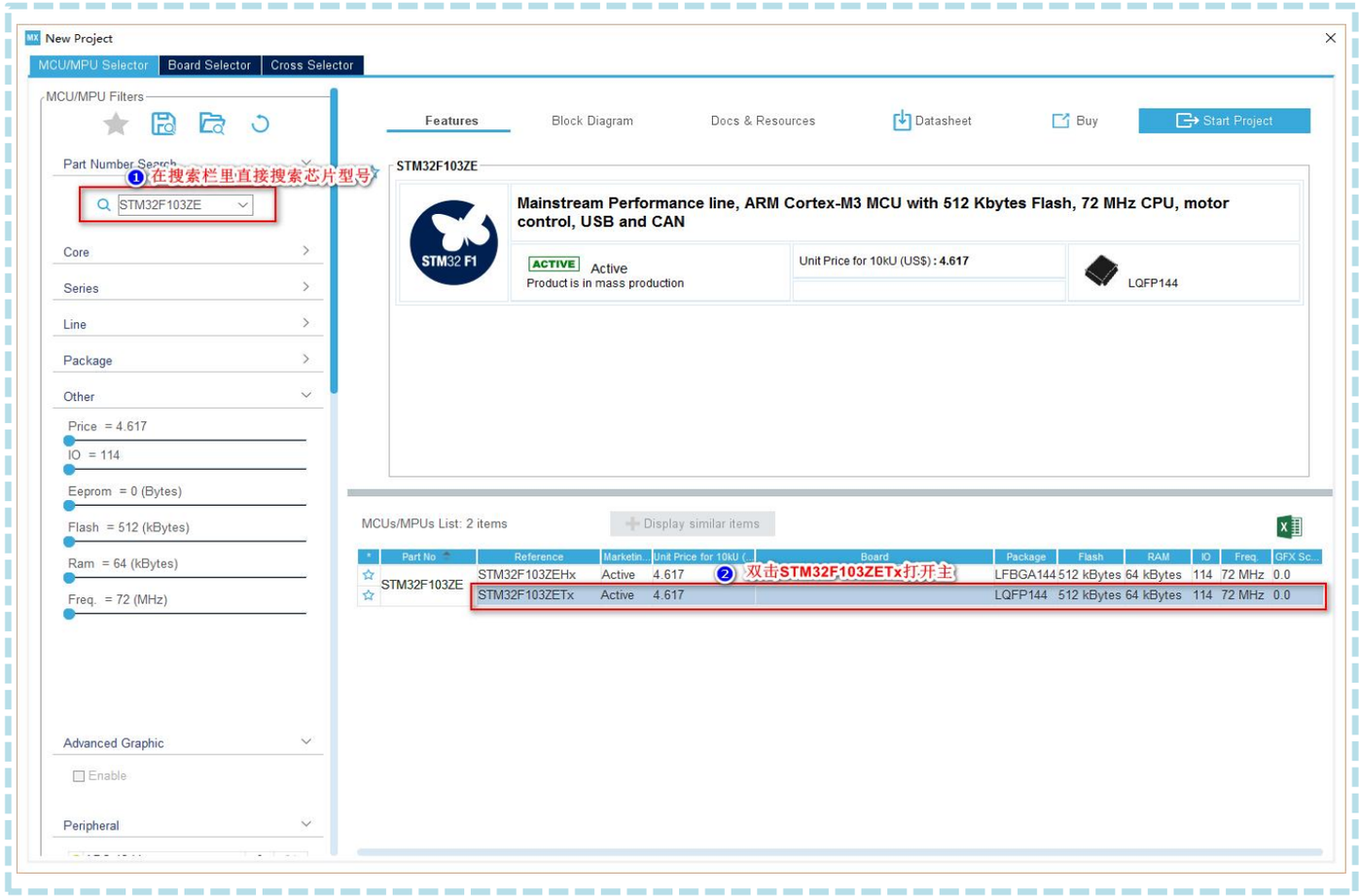
- 通过连接在ADC1的通道16上的STM32F103内部温度传感器获取芯片内部的温度；
- 利用DMA传输方式通过串口将采集到的温度数据显示出来。

10.4.2 HAL库ADC应用实例



软件设计——新建STM32CubeMX工程，选择MCU

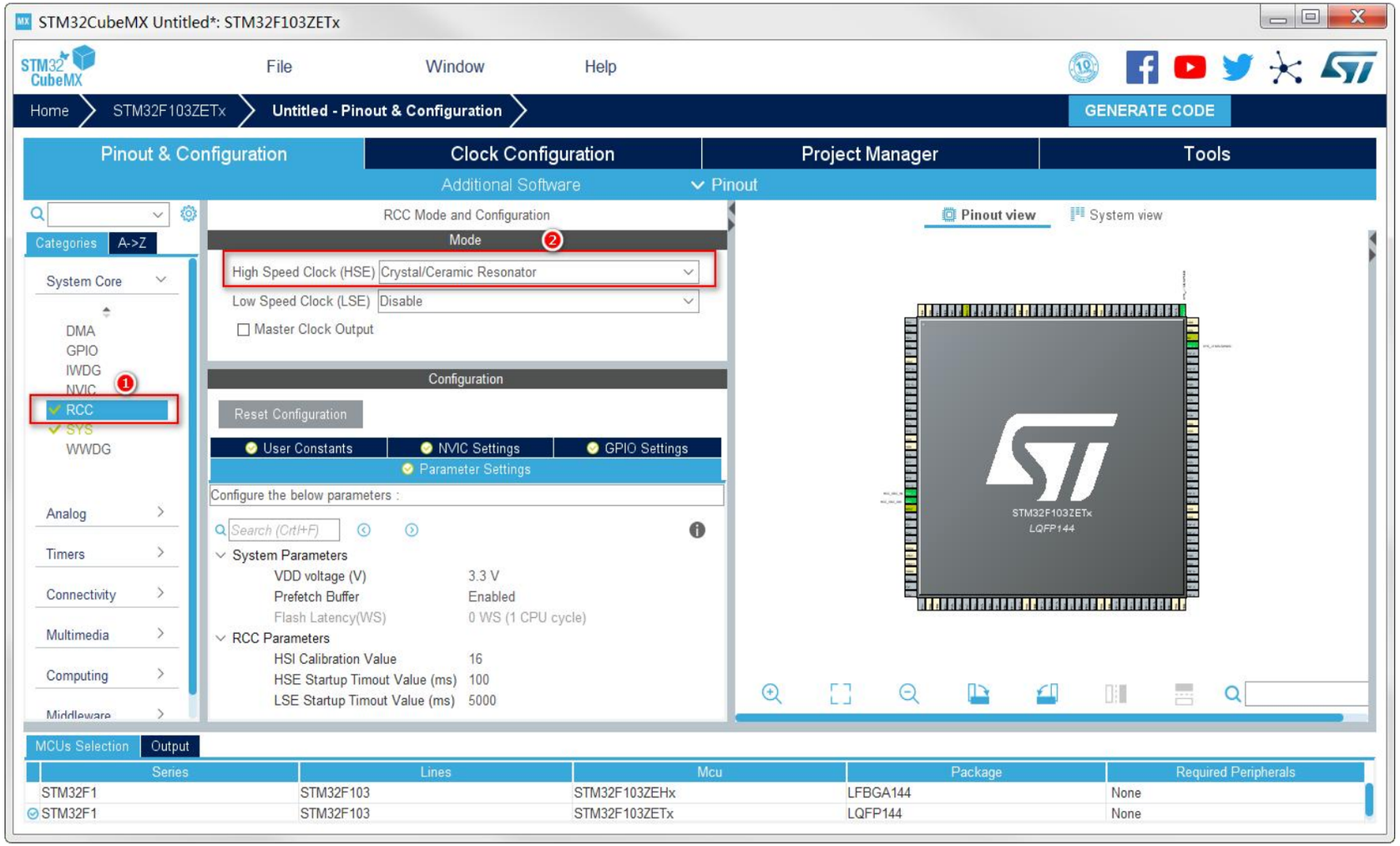
新建STM32CubeMx工程，选择MCU，这里选择STM32F103ZETx芯片，读者可根据自己的目标板选择相应的芯片



软件设计——STM32CubeMX功能参数配置

RCC配置

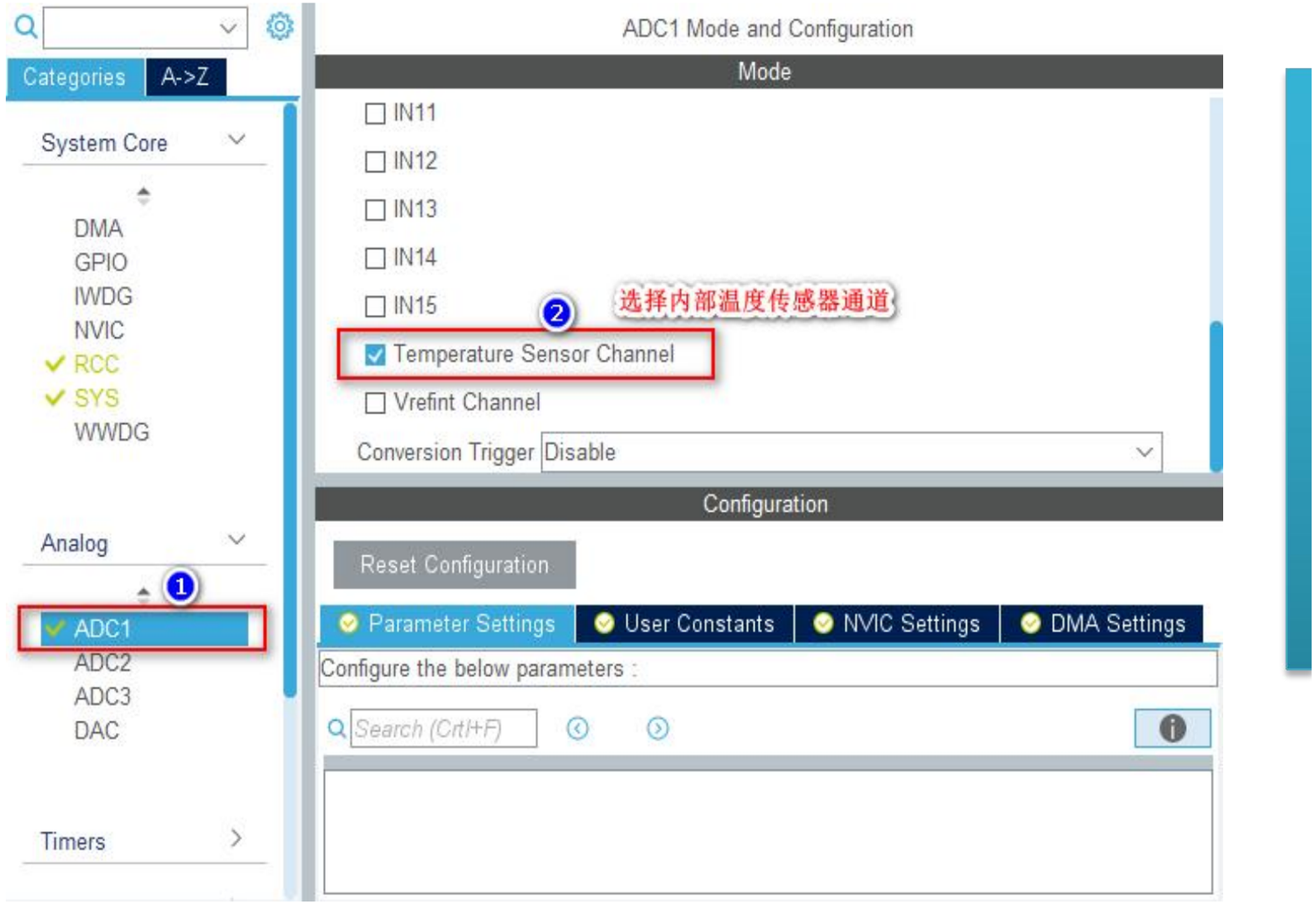
HSE选择“Crystal/Ceramic Resonator”(晶振/陶瓷谐振器)，LSE选择“Disable”



软件设计——STM32CubeMX功能参数配置

ADC配置——设置ADC1的通道

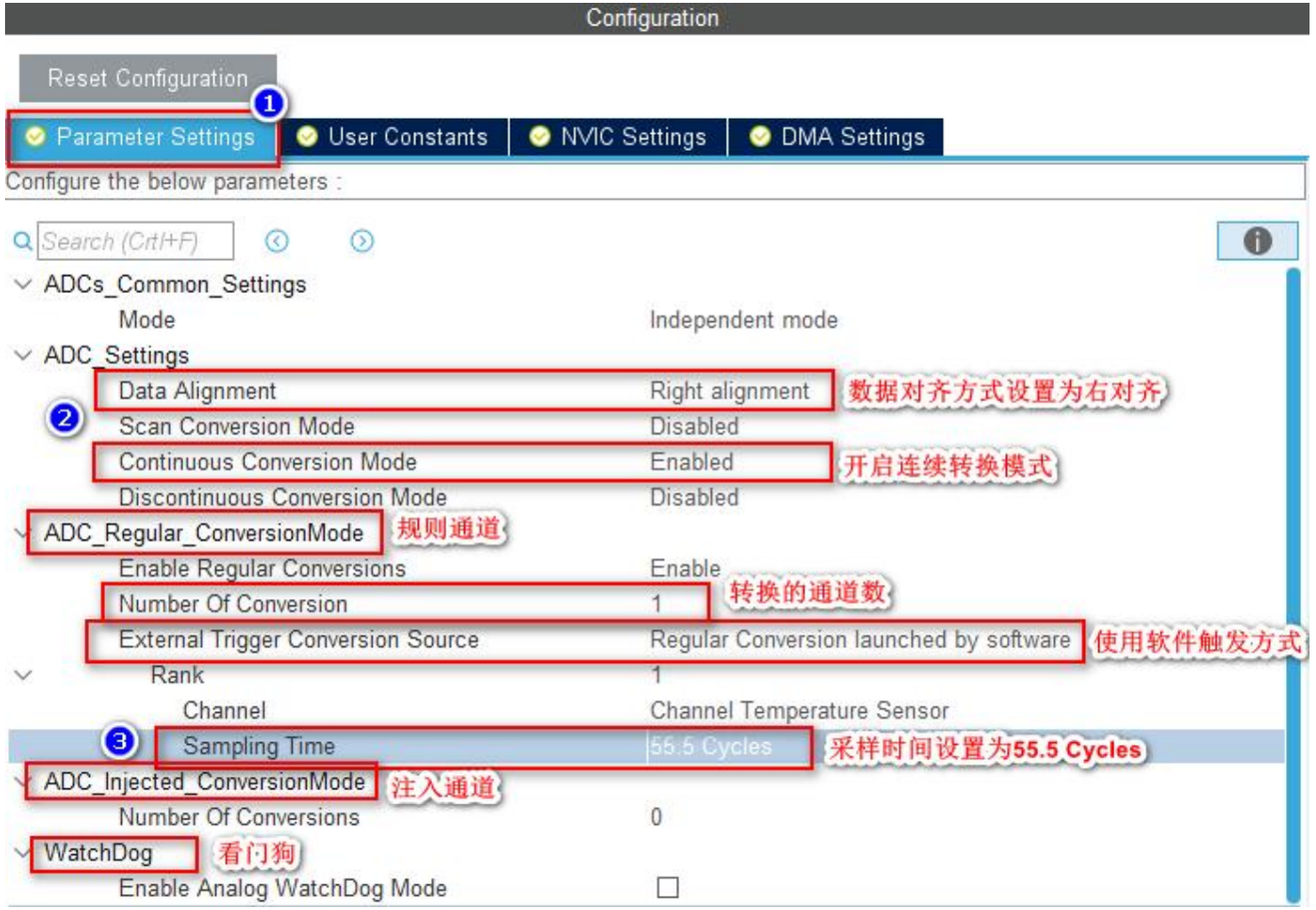
本实例采用ADC1的“Temperature Sensor Channel”通道(即连接在ADC1_IN16)



软件设计——STM32CubeMX功能参数配置

ADC配置——配置ADC1的参数

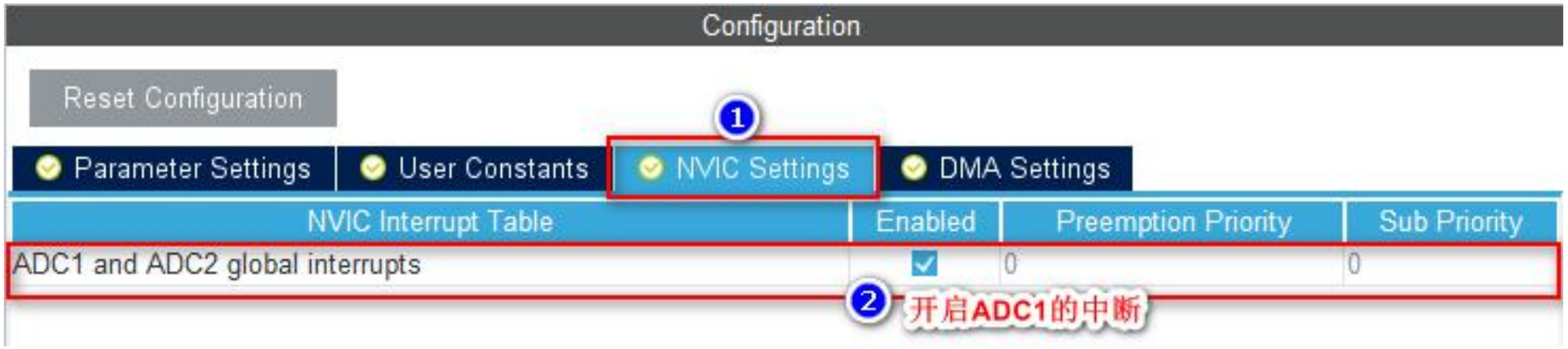
在ADC1参数配置“Configuration”选项配置页的“Parameter Settings”中对ADC1的相关参数进行设置，本例配置ADC1为独立模式，设置数据对齐方式为右对齐，禁止扫描转换模式，使能连续转换模式，在“Rank”项中设置采样周期为55.5Cycles



软件设计——STM32CubeMX功能参数配置

ADC配置——开启ADC1的中断

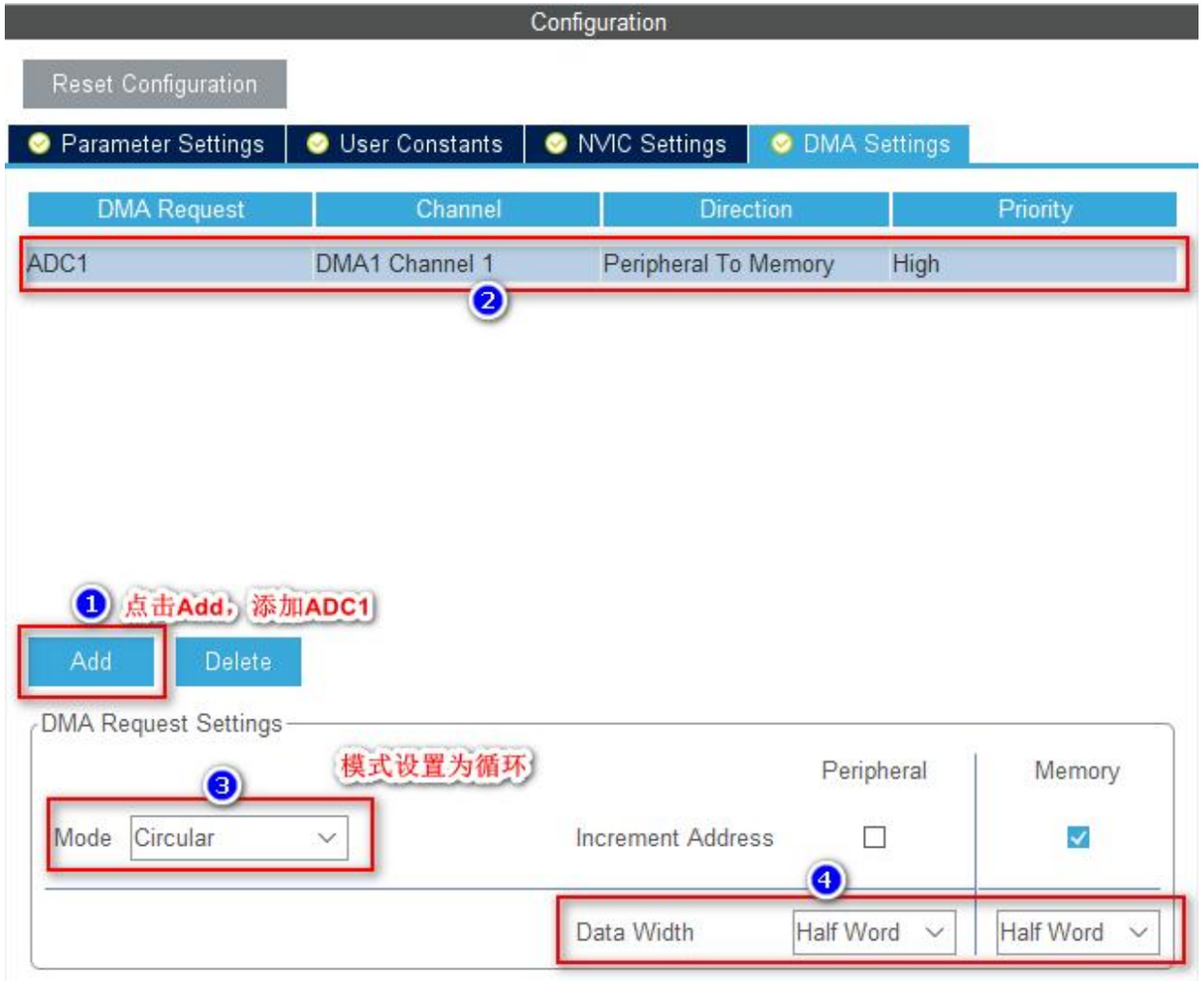
在ADC1参数配置“Configuration”选项配置页的“NVIC Settings”中开启ADC1的全局中断



软件设计——STM32CubeMX功能参数配置

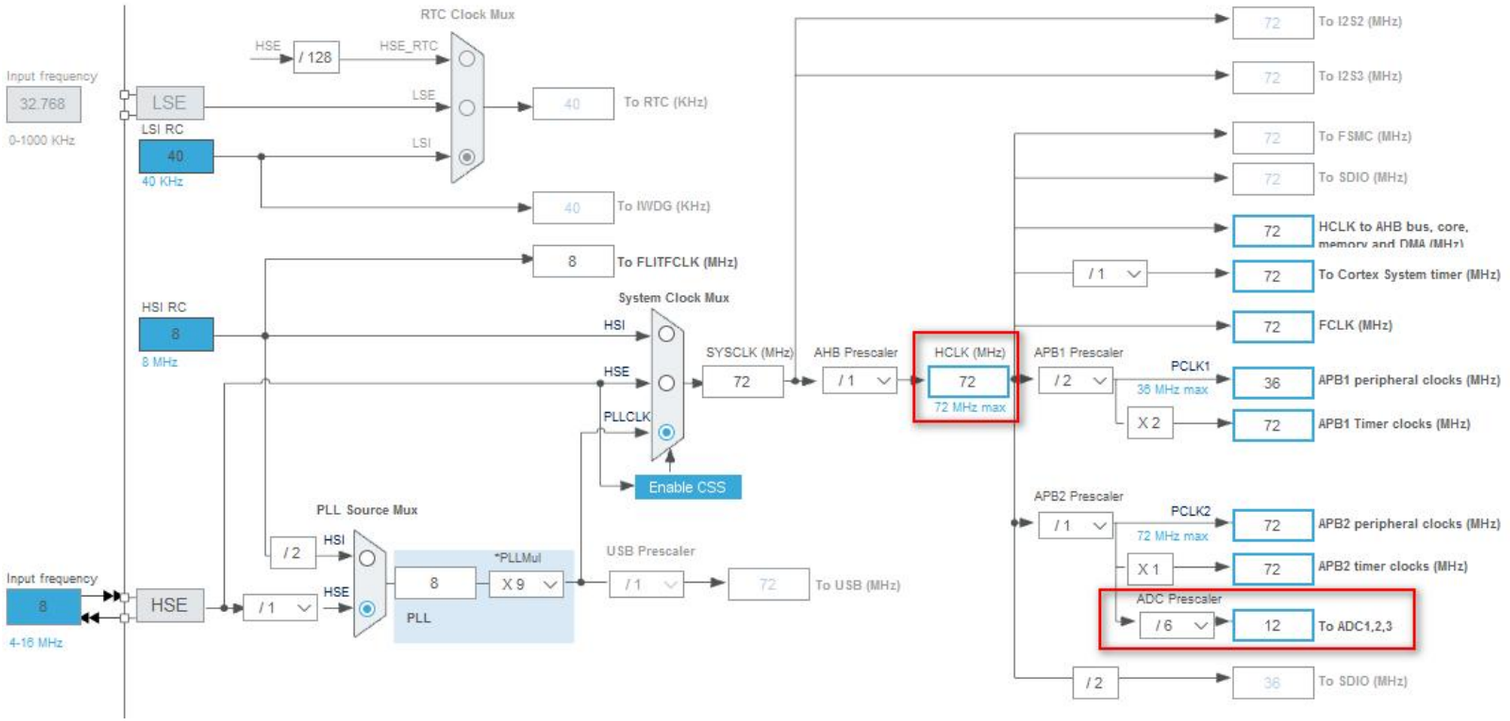
ADC配置——添加ADC1的DMA传输

在ADC1参数配置“Configuration”选项配置页的“DMA Settings”中，单击“Add”，添加ADC1的DMA传输方式，设置其优先级“Priority”为“High”，“Mode”选择为“Circular”，“Data Width”设置为“Half Word”



软件设计——STM32CubeMX功能参数配置

设置STM32时钟系统和ADC的时钟

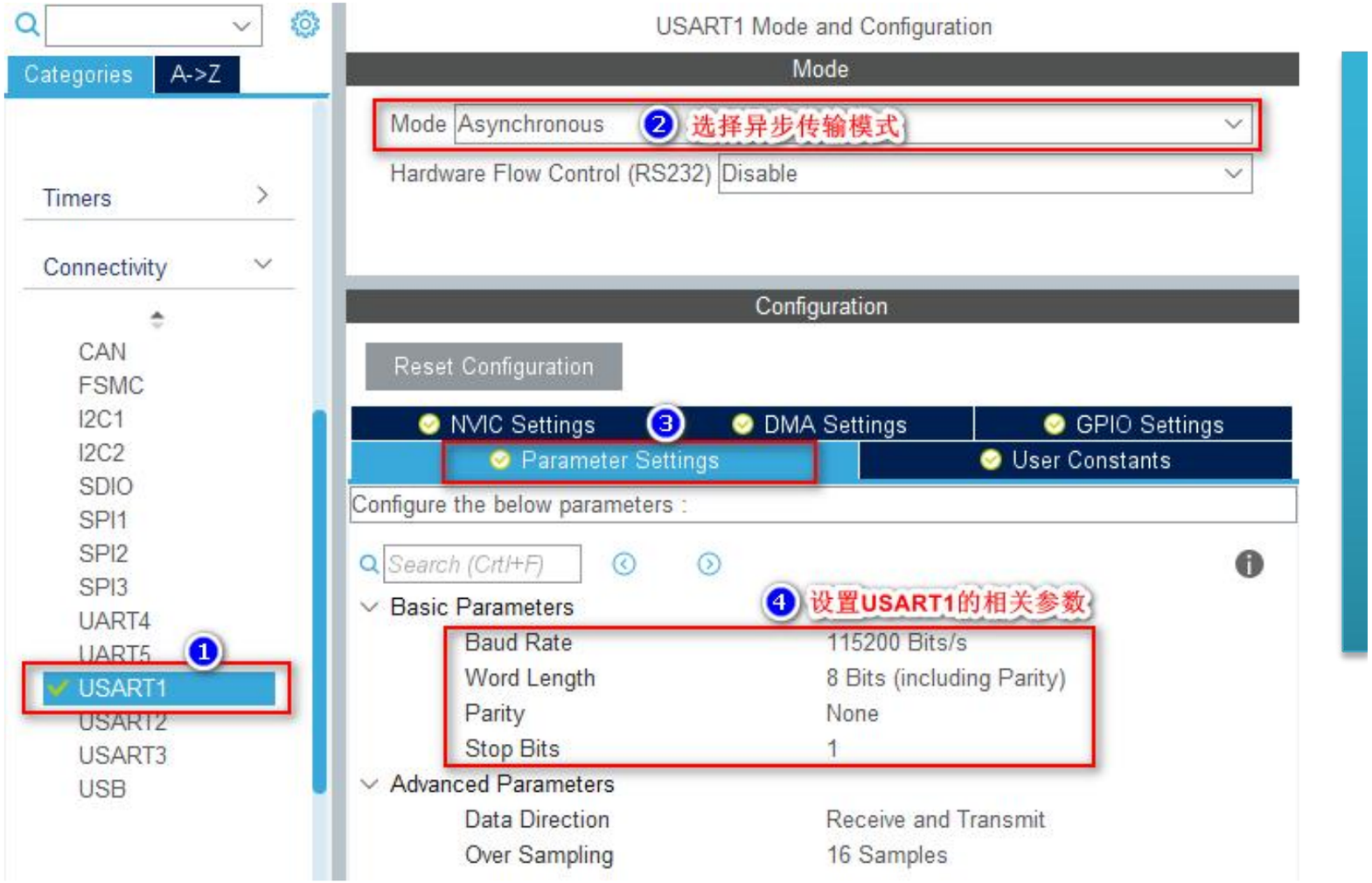


- 使用的外部晶振为8MHz
- 配置系统时钟为72MHz
- APB2为72MHz
- APB1为36MHz
- ADC为12MHz

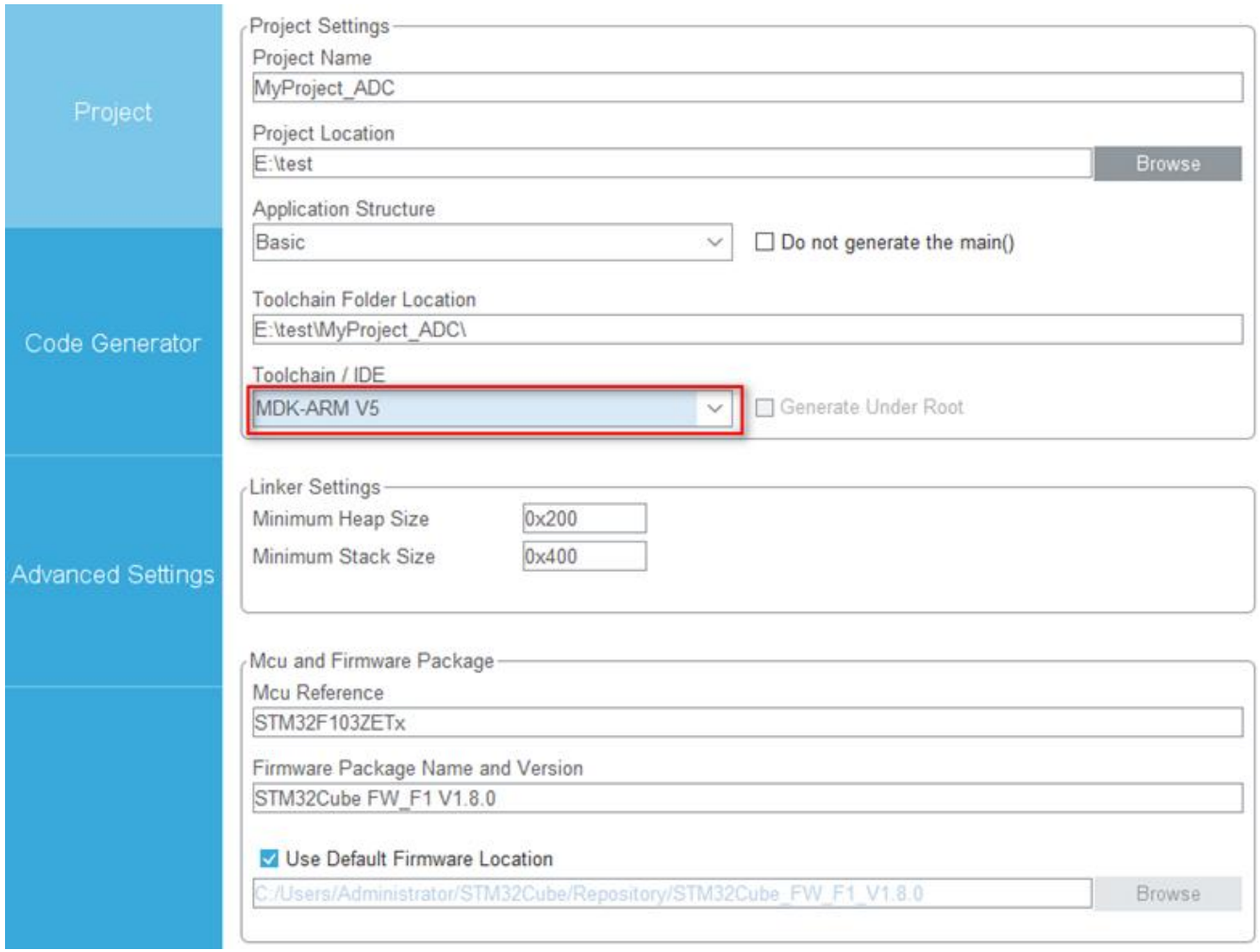
软件设计——STM32CubeMX功能参数配置

串口USART1配置

- 设置USART1为“Asychronous”模式
- 波特率设置为115200，数据位数为8
- 校验位选择无校验
- 停止位为1位



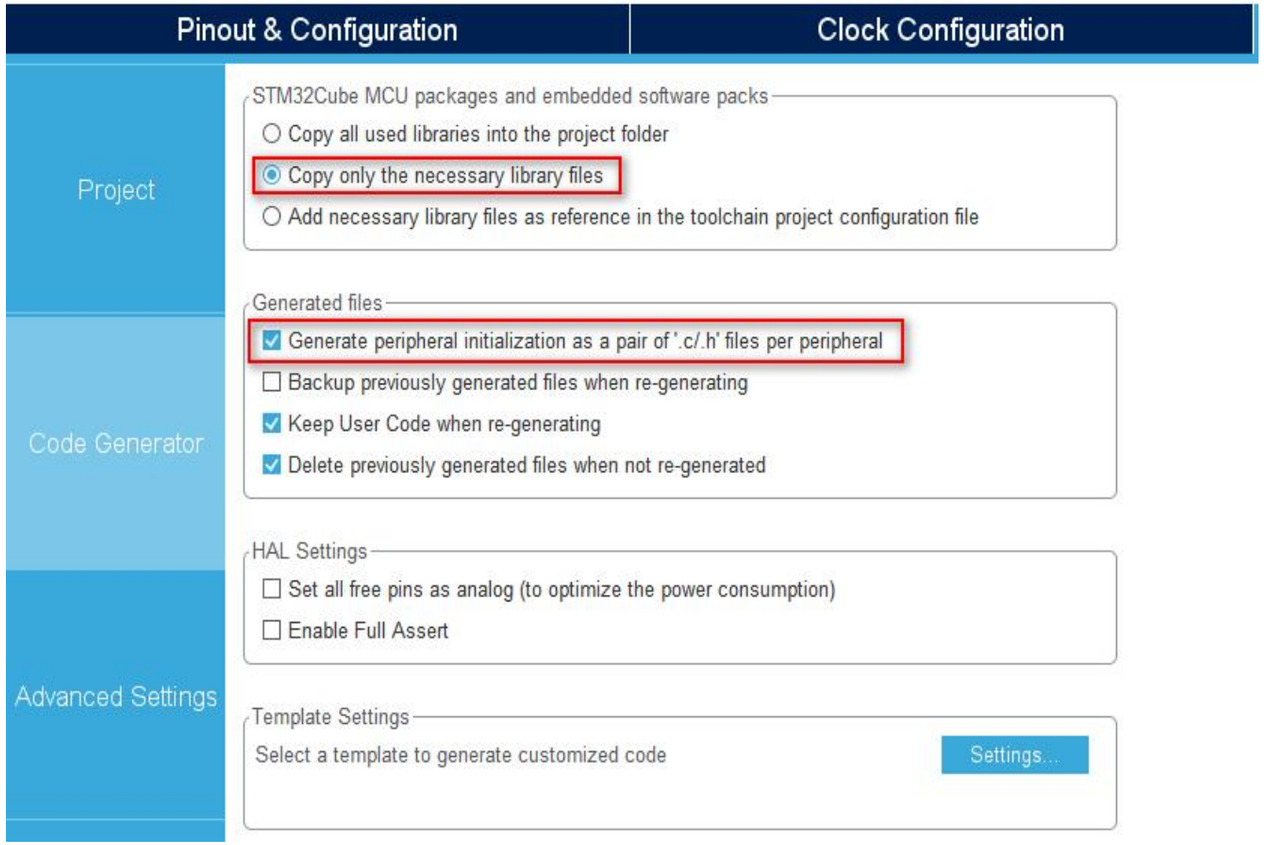
软件设计——生成工程代码



配置keil工程名称和存放位置

配置工程名称和存放位置，输入“Project Name”为“MyProject_ADC”，“Toolchain/IDE”选择“MDK-ARM V5”

软件设计——生成工程代码



在“Code Generator”选项栏中找到“Generated files”框，勾选“Generate peripheral initialization as a pair of ’.c/.h’ files per IP”，将外设初始化的代码设置生成为独立的.c源文件和.h头文件。

软件设计——编写应用程序

- 在main.c文件中的添加代码。先定义两个数组DMA_RxBuffer和DMA_TxBuffe，分别用于存放接收数据和发送数据，将此代码放在main.c文件中的/* USER CODE BEGIN PV */和/* USER CODE END PV */之间。

```
/* USER CODE BEGIN PV */
//定义V25和Avg_Slope两个宏
#define  V25  1.43
#define  Avg_Slope  0.0043

uint16_t  ADC_Converted_Value;
//用于存放AD转换的结果
uint16_t  MCU_Temperature;
 //用于存放芯片内部的温度值
/* USER CODE END PV */
```

软件设计——编写应用程序

- 然后在int main(void)函数中的/* USER CODE BEGIN 2 */和/* USER CODE END 2 */之间添加代码：

```
/* USER CODE BEGIN 2 */
/* ADC1校准*/
HAL_ADCEx_Calibration_Start(&hadc1);
/*启动ADC1的DMA转换 */
HAL_ADC_Start_DMA(&hadc1,(uint32_t*)
&ADC_Converted_Value,sizeof(&ADC_C
onverted_Value));
/* USER CODE END 2 */
```

软件设计——编写应用程序

- 在while语句中的/* USER CODE BEGIN 3 */和/* USER CODE END 3 */之间添加代码

```
while (1)
{
    /* USER CODE END WHILE */
    /* USER CODE BEGIN 3 */
    HAL_Delay(1000);
    printf("----ADC+DMA+USART实验 -----
\n");
    MCU_Temperature = (V25-
ADC_Converted_Value*3.3/4095)/Avg_Slope
+25;
    printf("\r\n The MCU temperature = %3d
\r\n ",MCU_Temperature);
}
/* USER CODE END 3 */
```

软件设计——编写应用程序

- 在usart.c的/* USER CODE BEGIN 1 */和/* USER CODE END 1 */添加代码：

```
/* USER CODE BEGIN 1 */
//重定向fputc函数
int fputc(int ch, FILE *f)
{  HAL_UART_Transmit(&huart1, (uint8_t *)&ch, 1,
0xffff);
  return ch;
}
//重定向fgetc函数
int fgetc(FILE * f)
{  uint8_t ch = 0;
    HAL_UART_Receive(&huart1,&ch, 1, 0xffff);
    return ch;
}
/* USER CODE END 1 */
```

软件设计——编写应用程序

采用轮询方式通过ADC1将STM32F103芯片内部的温度传感器采集到的温度数据通过串口发送到PC

添加全局变量

```
#define  V25  1.43
#define  Avg_Slope  0.0043

uint16_t  ADC_Converted_Value  = 0;
    //用于存放A/D转换的结果
```

软件设计——编写应用程序

在while(1)中添加代码

```
/* USER CODE BEGIN 3 */
HAL_ADC_Start(&hadc1);//启动ADC装换
HAL_ADC_PollForConversion(&hadc1, 50);
     //等待转换结束，第二个参数为超时时间，单位为ms
     //判断转换完成标志位是否设置，HAL_ADC_STATE_REG_EOC为转换完成标志
位
if(HAL_IS_BIT_SET(HAL_ADC_GetState(&hadc1), HAL_ADC_STATE_REG_EOC))
{  ADC_Converted_Value = HAL_ADC_GetValue(&hadc1);
        //读取ADC转换数据，数据为12位
  printf("MCU Temperature : %.1f度\r\n",( (V25-ADC_Converted_Value*3.3/
4095)/Avg_Slope+25);
}
HAL_Delay(1000);
/* USER CODE END 3 */
```

本章小结

ADC基础理论知识

- 转换过程
- 主要技术参数

ADC模块的标准外设库接口函数及应用

- 接口函数
- 配置过程
- 应用实例



STM32的ADC模块

- 引脚、通道选择、中断、DMA请求、转换时间、数据对齐、转换模式、校准

ADC模块的HAL库接口函数及应用

- 接口函数
- 应用实例



The End!

